Applied Soft Computing 139 (2023) 110219

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Discrete komodo algorithm for traveling salesman problem

Gilang Kusuma Jati [a,b,*], Garry Kuwanto [a], Tahir Hashmi [c], Herman Widjaja [c]

[a] Customer Excellence, Growth Engineering, Tokopedia, Jakarta, 12950, Indonesia
[b] New Retail, Growth Engineering, Tokopedia, Jakarta, 12950, Indonesia
[c] CTO Office, Tokopedia, Jakarta, 12950, Indonesia

ARTICLE INFO

Article history:
Received 6 October 2022
Received in revised form 6 March 2023
Accepted 14 March 2023
Available online 20 March 2023

Keywords:
Discrete komodo algorithm
Swarm intelligence
Traveling salesman problem
Edge-construction operator
Edge-destruction operator
Priority queue

ABSTRACT

This work presents a Discrete Komodo Algorithm (DKA) to solve the traveling salesman problem (TSP). Inspired by the Komodo Mlipir Algorithm, a swarm intelligence algorithm based on the behavior of komodo dragons, DKA introduces an enhanced edge-construction operator and a novel edge-destruction operator to explore the discrete search space of TSP. To prioritize the exploration of potential edges, DKA uses a Priority Queue that stores all possible edges with their weights, and its counts appear on tour as its priority. The edges with smaller weights and counts have higher probabilities of being selected for the tour. DKA is tested with 45 different TSP instances from TSPLIB. Experimental results show that DKA guarantees produced the best-known solution for small test problems for up to 24 nodes, and the best solutions are within 6.92% of the known optimal solutions. Compared with some of the previous algorithms, DKA outperforms the recent state-of-the-art algorithms and is significantly better than classical metaheuristic algorithms by achieving a lower relative error.

© 2023 Elsevier B.V. All rights reserved.

Code metadata

Permanent link to reproducible Capsule: https://doi.org/10.24433/CO.8050787.v1

1. Introduction

The Traveling Salesman Problem (TSP) is one of the most well-known combinatorial optimization problems. TSP is defined as follows: given a set of prescribed cities and a known cost of travel (e.g., distance, time, money) between each possible pair of cities, a salesman must find the best possible way to visit all the cities exactly once and return to the starting point while minimizing the travel cost. The challenge is to arrange the cities in the best possible order.

The Austrian mathematician Karl Menger started the work of TSP in 1932 and called TSP *messenger problem* (Botenproblem) [1]. A 1955 paper by Morton and Land indicates that statisticians may be familiar with TSP as *the mean minimum distance problem* [2]. Before this problem became widely known as TSP, most people believed that Julia Robinson was the one who published the first report using TSP as a name for this optimization problem in the United States in 1949 [3]. The TSP has many real-world parallels such as machine sequencing, and scheduling [4,5], cellular manufacturing [6,7], mixed windy rural postman problems, arc routing problems as a generalized TSP with directed edge [8], frequency assignment problem in a communication network [9,10], data analysis in psychology [11], analysis of the structure of crystals in X-ray crystallography [12], overhauling gas turbine engines [13], material handling in a warehouse order-picking problems [14], and wallpaper cutting [15]. At Tokopedia, TSP is used to model the daily visit recommendation for the sales agents, called *sobat*, to visit their clients. Although a modern-day traveling salesman's business tour may not seem too complex regarding route planning, the TSP generally represents a typical "hard" combinatorial optimization problem.

Researchers usually solve optimization problems like TSP by using deterministic and probabilistic algorithms. The most straightforward and well-known deterministic solution is the brute-force search, which generates all permutations (ordered combinations), evaluates the quality of each solution, and chooses the cheapest one. The running time for this approach is $O(n!)$, where n is the number of nodes that will be visited. As a result, the execution time to find the optimal solution will grow factorial

The code (and data) in this article has been certified as Reproducible by Code Ocean: (https://codeocean.com/). More information on the Reproducibility Badge Initiative is available at https://www.elsevier.com/physical-sciences-and-engineering/computer-science/journals.

* Corresponding author at: Customer Excellence, Growth Engineering, Tokopedia, Jakarta, 12950, Indonesia.
E-mail addresses: gilang.jati@tokopedia.com, gilang@ui.ac.id (G.K. Jati), garry.kuwanto@tokopedia.com, gkuwanto@bu.edu (G. Kuwanto), tahir.hashmi@tokopedia.com (T. Hashmi), herman.widjaja@tokopedia.com (H. Widjaja).

https://doi.org/10.1016/j.asoc.2023.110219
1568-4946/© 2023 Elsevier B.V. All rights reserved.

to the size of the given input. If n is the number of nodes to visit, there would be $(n-1)!/2$ feasible solutions and take $O(n)$ to evaluate each solution. Another faster deterministic algorithm is a dynamic programming approach called the Bellman–Held–Karp algorithm that solves the problem in $O(2^n n^2)$ [16–18], branch-and-bound [19], branch and cut [20,21], branch and price [22] and cutting planes [23,24]. These exact algorithms generally find an acceptable solution for small and medium-sized problems. However, the computation time for solving very large TSP using these exact algorithms would not be acceptable. Researchers have already proved that no scheme or algorithm currently exists to find the exact solution in polynomial time [25,26]. Many researchers turn to heuristic and metaheuristic algorithms that can find a satisfactory or near-optimal solution within an acceptable computation time.

Although the optimal solution is not guaranteed, heuristic and metaheuristic approaches can find high-quality solutions at a reasonable time [27]. Some of the most famous heuristic algorithms for TSP are Simulated Annealing (SA) [28] and Genetic Algorithm (GA) [29–31]. Among all the metaheuristics algorithms, the nature-inspired methods based on popular swarm intelligence achieve an effective solution in a reasonable time. A few of these types of algorithms are Particle Swarm Optimization (PSO) [32,33], Ant Colony Optimization (ACO) [34–38], Artificial Bee Colony (ABC) [39,40], and others successfully applied to solve TSP. A hybrid approach based on two heuristic or metaheuristic algorithms, such as SA and TS [41], a Hierarchic Approach (HA) of ACO and ABC [42], PSO and ACO [43], has shown a better result than individual approaches in solving TSP. All the metaheuristic algorithms nowadays look similar. They depend on two concepts: the movement of an individual and the interaction among individuals within a population. Nevertheless, the recent ones were developed with some improvements, resulting in better and faster performance than previous methods.

This study focuses on solving TSP with one of the recently proposed nature-inspired algorithms known as the Komodo *Mlipir* Algorithm (KMA) [44]. As this research started, no literature was available on KMA to solve discrete optimization problems. This study is the first in which KMA addresses the TSP. This study aims to attract readers to this efficient metaheuristic for solving TSP and other discrete optimization problems. The result obtained by the discrete version of KMA for TSP also emphasizes the future applicability of KMA in efficiently solving complex discrete optimization problems. This study incorporates several metaheuristic algorithms for comparison, such as the Black Hole Algorithm (BH) [45], Discrete State Transition Algorithm (DSTA) [46,47], Discrete Tree-Seed Algorithm (DTSA) [48], and Discrete Jaya Algorithm (DJAYA) [49]. For a rigorous and fair comparison, two statistical tests, the Friedman and Holm tests, are performed in addition to conventional tests based on best result, mean result, standard deviation, and relative error.

The highlights of the main contributions from this study are listed below:

- DKA is a novel discrete and permutation-coded optimization algorithm for the traveling salesman problem.
- DKA uses the total different edges between two solutions as a discrete distance so that DKA has visibility of potential edges as an exploitation area between two solutions.
- This study introduces an edge-construction operator, an enhanced version of DFA's edge-based movement [50].
- This study creates a novel edge-destruction operator to increase a discrete distance between two solutions in the discrete search space.
- DKA uses a priority queue to store all possible edges with edge weight, and the counter of the edge appears on tour as its priority. This priority queue enables DKA to find promising edges faster.

In the following sections of this study, Section 2 gives a literature review and discusses the previous research on metaheuristics for solving TSP. Section 3 briefly describes the TSP, and Section 4 gives an overview of the original KMA. Section 5 outlines the implementation of DKA for solving TSP. Section 6 shows the experimental results, convergence behavior, parameter analysis, comparison with other metaheuristic algorithms, and statistical analysis. Finally, Section 7 concludes the article by providing its future scope.

2. Literature review

KMA is a population-based nature-inspired algorithm proposed by Suyanto in 2022 to solve continuous optimization problems [44]. Unlike other metaheuristic algorithms, KMA divides its population into three groups of individuals based on their fitness (quality): high-quality big males, a single middle-quality female, and low-quality small males, where each group has a different movement scheme to exploit and explore the search space. Three different levels of exploitation and exploration of its movement strategies enable KMA to reach global optima for 7 out of 13 benchmark functions with fewer than 5000 evaluations, which is significantly better and faster than previously developed metaheuristic algorithms.

TSP is one of the discrete optimization problems studied extensively since 1930, both in industry and academia. TSP has caught the attention of many researchers due to its wide range of real-world applications [51]. As one of the NP-Complete problems [52], it is also frequently used to validate the performance of new discrete optimization algorithms. The literature shows many algorithms developed to solve TSP, such as exact approaches, heuristics, and metaheuristic algorithms. The most well-known approach is the brute-force algorithm, which generates all possible solutions, evaluates each solution, and selects the best one. The search space of the combination explosion of the brute force approach grows factorially with the size of the given input due to its time complexity of $O(n!)$, where n is the number of nodes that will be visited. Even though this exact approach is guaranteed to provide the optimum solution, the running time of this approach makes this approach less preferable when the best solution is not required.

In 1962, Held and Karp [16] formulated TSP with dynamic programming together with other sequencing problems: a scheduling problem and an assembly-line balancing problem. In the same year, Bellman [17] also formulated TSP in dynamic programming terms and could resolve TSP computationally for up to 17 nodes. In Held and Karp [18] another work, they explored a new approach to solve symmetric TSP with a minimum spanning tree. Since then, people have known this dynamic programming approach as the Bellman–Held–Karp algorithm. This algorithm can provide an optimum solution in exponential time $O(2^n n^2)$, significantly better than a brute-force algorithm's factorial performance $O(n!)$. It utilizes the distance matrix as an input so that it requires $O(n2^n)$ space to hold all computed values, while brute force needs only $O(n^2)$ space to store the graph itself. In 1986, Radharamanan and Choi [19] presented a branch and bound model with penalty tour building to solve TSP and transport routing problems. One year later, Padberg and Rinaldi [20] reported the solution to optimality of a 532-city symmetric TSP with the branch-and-cut algorithm. In 2004, Hernández-Pérez and Salazar-González [21] introduced a 0–1 integer linear model with the branch-and-cut algorithm for a routing problem called one-commodity pickup and delivery TSP. Banhart et al. [22] proposed a branch and price method for column generation to solve huge integer programs and use this approach to solve large instances of the TSP. In 1985, Fleischmann [23] proposed a cutting plane procedure that can

produce an exact solution for the TSP for up to 120 nodes of classical problems and real road networks with up to 292 visiting places.

In the heuristic approach, Geng et al. [28] proposed an effective local search algorithm based on Simulated Annealing (SA) to solve the TSP using three kinds of mutations with different probabilities during its search and used greedy search techniques to speed up the convergence rate. In 2014, Ahmed [29] proposed an improved GA to solve TSP using four different operators: a 2-opt search, a hybrid mutation, a combined mutation operator, and sequential constructive crossover. The experimental result shows that an Improved GA outperformed a simple GA. Changdar et al. [30] presented a multi-objective GA for TSP in a fuzzy environment where cost and time are the objectives. They used a cyclic crossover and two-point mutation as the operator to generate new solutions. In another study, Wang [31] proposed GA with two local optimization strategies. The first method is the four vertices and three lines inequality to generate shorter Hamiltonian circuits. The second method reverses the local Hamiltonian paths with more than two vertices.

There are many metaheuristic methods in the literature for solving TSP. Mohemmed et al. [32] used Particle Swarm Optimizer (PSO) to solve the shortest path problem using a modified priority-based encoding incorporating a heuristic operator for reducing the possibility of loop-formation in the path-construction process. In another study, Vallade et al. [33] proposed a PSO-based approach to solve TSP. The original PSO is used to tune a set of parameters, which usually take their values in static search spaces. However, Vallade modified PSO to take the values in dynamic space.

In 1996, Dorigo et al. [34] proposed an Ant System (AS), a population-based algorithm inspired by cooperating agents of an ant colony. The AS guided each agent by an *autocatalytic* process directed by a greedy force. Another study by Dorigo and Gambardella [35] proposed an artificial ant colony for solving TSP where ants of the artificial colony can generate successively shorter feasible tours. In 2014, Sahana et al. [36] proposed another implementation of Ant Colony for TSP called High-Performance Ant Colony Optimizer (HPACO). They modified the pheromone update procedure, made multiple ants as a group, and applied the effect of the group update process on the edges of the TSP graph instead of making a single ant move randomly. The result shows that the HPACO has better performance than the conventional ACO. Uchida et al. [37] presented an efficient implementation of ACO using GPU, which could speed up its running time up to 22.11 times faster than CPU. Escario et al. [38] proposed an extended version of ACO called Ant Colony Extended (ACE) to solve TSP. They introduced a division task between two kinds of ants in ACE, the patrollers, and foragers, and a parameter to control the number of each ant during the searching process. ACE performs better than other algorithms in the comparison for almost every TSP test problem. In 2011, Karaboga et al. [39] successfully implemented Combinatorial Artificial Bee Colony (ABC) for TSP. Kıran et al. [40] proposed a discrete version of ABC with a neighborhood operator. The discrete ABC optimized its result with 2-opt and 3-opt heuristic approaches. Gündüz et al. [42] proposed a Hierarchic Approach (HA) based on two swarm intelligence: ACO and ABC. They used ACO as a path construction-based method to provide an initial solution for the path improvement-based method ABC. The results show that HA produces better quality solutions than individual approaches of ACO and ABC.

In 2018, Hatamlou [45] proposed the Black Hole algorithm to solve TSP, an algorithm inspired by the black hole phenomenon. The experimental results show that the BH algorithm can find high-quality solutions compared to GA, ACO, and PSO algorithms. Chunhua et al. [46] proposed a Discrete State Transition Algorithm (DSTA) to solve TSP with swap, shift, and symmetry as operators. X. Zhou et al. [47] formulated another DSTA to solve the unconstrained integer optimization problem. DSTA uses the same three state transformation techniques, swap, shift, and symmetry, to generate new candidate solutions. DSTA was compared with SA in this work, and the results show that its variant, DSTAI, has almost the same performance as the SA, while DSTAII outperformed SA. In 2020, Cinar et al. [48] proposed a discrete version of the Tree-Seed algorithm called DTSA to solve TSP. DTSA used transformation operators such as swap, shift, and symmetry [46, 47] to generate new solutions. Experimental results show that DTSA is a qualified and competitive solver of discrete optimization. One year later, Gunduz et al. [49] proposed DJAYA, a discrete version of the Jaya algorithm. DJAYA used eight transformation operators and applied a 2-opt heuristic on the best individual produced by DJAYA. The experimental results show that DJAYA is a highly competitive and robust optimizer for TSP.

The success of the KMA in solving the continuous optimization problem is the inspiration for the creation of this paper.

3. The traveling salesman problem

Mathematically, we can define TSP as such; Let $K_n = (V_n, E_n)$ be a complete undirected graph with $n = |V_n|$ nodes and $m = |E_n| = \binom{n}{2}$ edges. An edge e with endpoints x and y is denoted by $x - y$. We denote $\mathbb{R}^{E_n}$ to be the space of real vectors whose components are indexed by elements of E_n. The component of any vector $v \in \mathbb{R}^{E_n}$ indexed by $e = x - y$ is denoted by v_{x-y} or v_e.

Given an objective function $c \in \mathbb{R}^{E_n}$ that associates the distance of c_e with every edge of $e \in E_n$, TSP consists of finding a *Hamiltonian Cycle* (a cycle visiting every node once) such that its c length (the sum of all the length of the edges) is minimum.

Let $C = \{e_{i,j}\} \subset E_n$ be a cycle and a candidate solution to TSP, the fitness of that cycle can be calculated as the sum of all of its edge length, or

$$f(C) = \sum_{e \in C} c_e \tag{1}$$

Because of the assumption of having a completed graph, it is always guaranteed that a feasible solution exists. Another consequence of K_n being complete is that the number of Hamiltonian Cycles, i.e., the size of the search space of K_n is t, calculated using this formula.

$$t = \frac{(n-1)!}{2} \tag{2}$$

4. Original komodo *mlipir* algorithm (KMA)

KMA is a population-based algorithm inspired by the behavior of Komodo dragons proposed by Suyanto [44]. KMA split the population into three groups for three different search strategies. The first group is high-quality big males, which perform high exploitation and low exploration in a narrow area around the global optimum; the second group is one middle-quality female that either does exploitation or exploration with a fixed probability of 0.5; the last group is low-quality small males, that perform *mlipir* movement to search for a solution in a broad area. Using a portion parameter p from the interval (0, 1), KMA splits the population of n Komodo individuals into q big males, one female, and s small males by using the following formulas.

$$q = \lfloor (1-p)n \rfloor \tag{3}$$

$$s = n - q \tag{4}$$

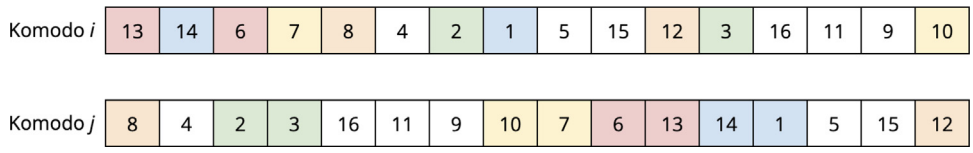


Fig. 1. The distance between two komodo i and j is five: edge $2-3$, $10-7$, $6-13$, $14-1$, and $12-8$.

KMA defines the movement of a big male k_i to produce a new position k_i' by using the formula.

$$w_{ij} = \begin{cases} r_1(k_j - k_i), & \text{if } f(k_j) < f(k_i) \text{ or } r_2 < 0.5 \\ r_1(k_i - k_j), & \text{otherwise.} \end{cases} \tag{5}$$

$$k_i' = k_i + \sum_{j=1}^{q} w_{ij}, \text{ where } j \neq i \tag{6}$$

where k_i and k_j are the ith and jth big males ($i \neq j$) and $f(k_i)$ and $f(k_j)$ are their fitness (objectives) respectively, r_1 and r_2 are random numbers in the interval [0, 1] in the normal distribution and q is the number of big males from Eq. (3).

KMA designates one middle-quality female to exploit or explore with a fixed probability of 0.5. If exploitation is selected, the female will generate two offspring, k_{il}' and k_{jl}', by mating with the highest quality big male using the formula in Eq. (7).

$$\begin{aligned} k_{il}' &= r_l \,.\, k_{il} + (1 - r_l) \,.\, k_{jl} \\ k_{jl}' &= r_l \,.\, k_{jl} + (1 - r_l) \,.\, k_{il} \end{aligned} \tag{7}$$

where k_{il} and k_{jl} are the lth dimension of the ith and jth Komodo as the highest quality big male and female, respectively, and r_l is the random number in the interval [0, 1] in the normal distribution for lth dimension. The best offspring, based on fitness, will replace the current female. If exploration is selected, KMA implements parthenogenesis (asexual reproduction) of a female Komodo by appending a small value to each female dimension.

$$(k_{i1}, k_{i2}, \ldots, k_{im}) \rightarrow (k_{i1}', k_{i2}', \ldots, k_{im}') \tag{8}$$

$$k_{ij}' = k_{ij} + (2r - 1)\alpha |ub_j - lb_j| \tag{9}$$

where m is the dimension of the problem, $(k_{i1}, k_{i2}, \ldots, k_{im}) \in [lb_j, up_j]$ is the vector element on m dimensions of a Komodo individual k, lb_j and up_j are the lower and upper bounds of jth dimension, r is a random value in the normal distribution, and α is the fixed radius of parthenogenesis with 0.1 as its value, which means the new solution can be generated in the radius of 10% of the search space from current komodo's state or position.

The small male ith Komodo attracted to the jth big male performs *mlipir* movement with a probability of *mlipir* rate d in the interval [0, 1] by the following formula.

$$w_{ij} = \begin{cases} \sum_{l=1}^{m} r_1(k_{jl} - k_{il}), & \text{if } r_2 < d \\ 0, & \text{otherwise.} \end{cases} \tag{10}$$

$$k_i' = k_i + \sum_{j=1}^{q} w_{ij}, \text{ where } j \neq i \tag{11}$$

where m is the dimension of the problem, k_{il} and k_{jl} is the lth dimension of ith small male and jth big male respectively, r_1 and r_2 are randomly generated numbers in [0, 1] using the normal distribution: r_1 defines the velocity of the movement while r_2 select the dimension to follow, l is randomly selected dimension with probability equal to the *mlipir* rate d to select $1, 2, \ldots, m$ dimension of the big male, and q is the number of big males.

Finally, KMA designs its population to be adaptive during evolution based on its best-so-far fitness in the last two iterations. KMA defines the new population size n' using Eq. (12).

$$n' = \begin{cases} n - a, & \text{if } \delta f_1 > 0 \text{ and } \delta f_2 > 0 \\ n + a, & \text{if } \delta f_1 = 0 \text{ and } \delta f_2 = 0 \end{cases} \tag{12}$$

$$\begin{aligned} \delta f_1 &= \frac{|f_1 - f_2|}{f_1} \\ \delta f_2 &= \frac{|f_2 - f_3|}{f_2} \end{aligned} \tag{13}$$

where a is the number of individuals to delete or generate, δf_1 and δf_2 are the fitness differences of two successive ith and $(i-1)$th generations, respectively.

5. Proposed discrete komodo algorithm

We proposed the Discrete Komodo Algorithm (DKA) to solve TSP with KMA as an inspiration. There are some modifications needed; there are:

- DKA uses permutation representation to represent a solution and uses the total distance of the tour as the fitness of individuals by using Eq. (1).
- DKA uses the total different edges between two solutions as a discrete distance (see Fig. 1).
- DKA initializes its population with a random permutation, except the first individual is created by the nearest neighbor tour like DTSA [48], and DJAYA [49].
- DKA uses enhanced edge-construction and novel edge-destruction operators to define attraction and distraction between two komodo as exploitation movement.
- To perform exploration movement within TSP discrete search space by small males, DKA uses the insertion operator and the 2-opt swap operator.
- To select potential edges to construct, DKA uses Priority Queue to store all possible edges with edge-weight, and the counter of edge appears on tour during iteration as its priority.

5.1. Edge-construction operator

This operator is used to construct an edge into the current tour without destroying existing similar contiguous edges on another better tour so that the discrete distance between two solutions is reduced. This operator works with a *segment* to construct an edge to existing solutions. Fig. 2 shows the identified four segments when komodo i wants to construct the edge $2-3$, the edge that exists on komodo j but does not exist on komodo i (see Fig. 1). The detail regarding how to build the segments and the original DFA's edge-based movement as an inspiration for this operator can be found at [50].

DFA generates four possibilities by combining S_x and S_y, while DKA extends to eight possibilities by including S_a and S_b to provide new solutions. By combining these four segments, the eight possibilities are

a. $S_a + S_b + S_x + S_y$
b. $rev(S_a) + S_b + S_x + S_y$
c. $S_a + rev(S_b) + S_x + S_y$

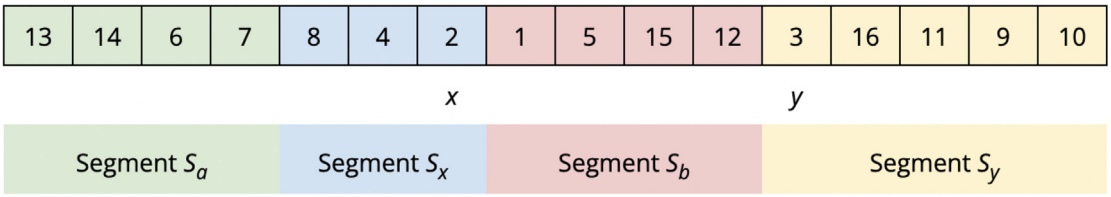


Fig. 2. Identified segments in komodo i to construct edge $2-3$.

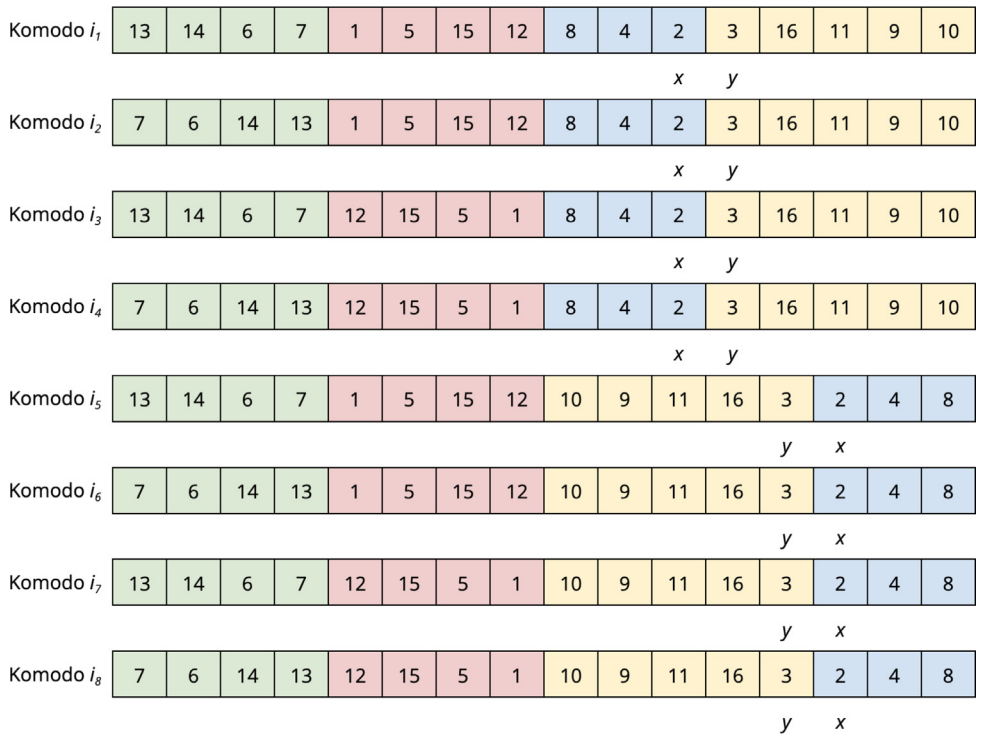


Fig. 3. Merging process of four segments to produce eight new solutions.

d. $rev(S_a) + rev(S_b) + S_x + S_y$
e. $S_a + S_b + rev(S_y) + rev(S_x)$
f. $rev(S_a) + S_b + rev(S_y) + rev(S_x)$
g. $S_a + rev(S_b) + rev(S_y) + rev(S_x)$
h. $rev(S_a) + rev(S_b) + rev(S_y) + rev(S_x)$

rev is the function to reverse the segment (array or sub-array). If segment S_n has $1-5-15-12$ as its elements, reversing the segment will get $12-15-5-1$ as a result. The reversion of a segment is required to ensure that the selected edge to construct (edge $2-3$) appears in komodo i. Executing all eight possibilities of merging these segments will yield eight new solutions.

Observe that when the length of S_a is equal to 1, then $rev(S_a) = S_a$, which will result in combinations a and b; c and d; e and f; g and h will result in the same tour, thus resulting in 4 unique new tours. And conversely, the length of S_b equal to 1 will result in combinations a and c; b and d; e and h; f and h being the same tour. In a special case when the length of S_a and S_b are both equal to 1, there will be only 2 unique tours, which are

a. $S_a + S_b + S_x + S_y$
b. $S_a + S_b + rev(S_y) + rev(S_x)$

Fig. 3 shows the merging result of the identified four segments S_x, S_y, S_a, and S_b. Komodo i_1 until komodo i_8 are the results of applying these eight possibilities, yielding eight new solutions. Then, we can choose randomly from these eight new solutions or choose the best fitness among these eight new solutions.

5.2. Edge-destruction operator

We build this novel operator to simulate the distraction movement of komodo in discrete search space to solve TSP. We design the edge-construction operator to construct an edge into the current tour and return the best solution from eight possibilities. As opposed to constructing a new edge, we design the edge-destruction operator to remove a particular edge that exists on the current tour but not on the other one. KMA reflects the edge-destruction phenomenon in big males' movement when distraction occurs in Eqs. (5) and (6), such that the distance between two solutions (komodo) increases.

The following procedure illustrates the edge-destruction operator. First, let there be two solutions, komodo i and komodo j. We select one from the similar edges in both solutions to remove from komodo i. Let edge $x-y$ be the selected edge. We first find the position (index) of node x and y in komodo i. Without loss of generality, we assume the index of node x is less than the index of node y, i.e., the node x comes before node y in the tour. Then, similarly to edge construction, we create three segments: the longest common subsection to the left of node x, denoted as S_x; the longest common subsection to the right of node y, denoted as S_y; and the remaining segment denoted as S_m. After identifying

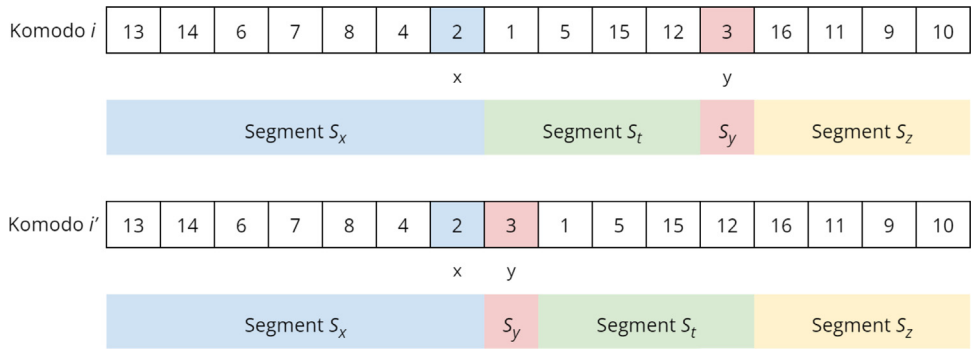


Fig. 4. Komodo i perform an Insertion operator for edge $2-3$.

the three segments, we merge them so nodes x and node y do not appear adjacent. Thus achieving the objective of removing the edge $x-y$ from komodo i. This operator generates six possible combinations of the segments, which are:

- $S_x + S_m + S_y$
- $S_x + S_m + rev(S_y)$
- $S_x + rev(S_m) + S_y$
- $S_x + rev(S_m) + rev(S_y)$
- $rev(S_x) + S_m + S_y$
- $rev(S_x) + rev(S_m) + S_y$

rev is the function to reverse the segment (array or sub-array).

With the current design of edge destruction, there is a known limit. It will fail to remove the desired edge if both segments S_x and S_y consist of only one node. A partial solution for this issue is taking one extra node to either S_x or S_y. The edge correlated to this extra node could be whether the edge exists on the other tour. Adding one extra node will ensure that at least one of the six solutions from the edge-destruction operation will remove the desired edge.

5.3. Priority queue

The DKA implementation uses a Priority Queue (PQ) to store all possible edges where each edge has a Priority Value (PV). DKA calculates the PV of the particular edge from its normalized edge weight and a counter to count how often DKA constructs this edge on candidate solutions during iteration. We design this PQ to pop the edge with the lowest PV as the priority. The lower the PV, the higher the priority. In other words, DKA sorts the elements/edges in PQ in ascending order, which means the DKA will prefer the shortest edge, which the fewest tour has used. DKA defines the PV of a particular edge e by using the formula

$$PV_e = norm(w_e) + cnt(e) \tag{14}$$

where $norm(w_e)$ is the normalized weight and $cnt(e)$ is the counter of edge e used or constructed during iteration. The $norm(w_e)$ has a fixed value while the value of $cnt(e)$ is dynamic, which makes the value of PV of edge e could change from one iteration to the next iteration. To keep the effect of weight and counter not varying between different problem sets, we use a normalized weight which stores the weight into a normalized value in the range of 0 to a normalized scale parameter $\mathcal{S}$, which we can see the mathematical formula for the calculation below.

Let $\mathcal{S}$ be the normalized scale used and let e be some edge, and e_{max} and e_{min} be, respectively, the edges with the longest and shortest weight in the whole graph, and w_e be the weight of particular edge e, then the normalized weight is as follows.

$$norm(w_e) = \mathcal{S} \cdot \frac{w_e - w_{e_{min}}}{w_{e_{max}} - w_{e_{min}}} \tag{15}$$

Since TSP is a minimization problem, an optimal solution for TSP depends heavily on the distance between edges. While we can use a greedy approach, it is not guaranteed to provide an optimum solution. So PQ plays an essential part in DKA to be slightly greedy but still be able to explore other edges proportionally to its weight. The PQ helps DKA prioritize to explore edges with shorter weights than longer ones while still allowing for the longer edges to be selected during iteration after some time. The longer edges will have higher priority (lower PV) to be selected and constructed after the counter, and the weight of the shorter edges exceeds the longer ones.

5.4. Exploration movement

The *mlipir* movement of small males in the KMA is a low-intensification (exploitation) high-exploration (LIHE) search, which means that it is a "guided" local search where the small males will take a random big male position as a guide to doing a LIHE search. To adapt this in DKA, we propose two kinds of algorithms to help with the local search: the insertion operator which is illustrated in Fig. 4 and the 2-opt swap operator [53], which is illustrated in Fig. 5. While we do not use the edge selection method (optimization) from those papers, we use their method of swapping to add desired edges to the tour. Both operators will ensure the desired edge exists in the new tour while creating some new edges. It is worth mentioning that the 2-opt swap operator differs from the 2-opt algorithm. 2-opt swap is only a swapping mechanism to connect the desired edge. In comparison, the 2-opt algorithm tries to remove all crossings on tour by performing a 2-opt swap if the swap results in a better/shorter tour.

DKA introduces a new parameter called Small Male Exploration Probability ($smep$) to control the movement of small males, whether they need to perform exploration or attraction movement to big males. DKA will make a random integer from 0 until 10, representing the probability of small males in DKA to explore edges from PQ or an attraction to big males with the edge-construction operator defined in Algorithm 3. Zero means DKA will never perform an exploration movement and will always perform an attraction movement to big males. Five means 50% movement of small males will perform an exploration movement. While ten means 100% movement of small males will perform an exploration movement and 0% chance of attraction movement to big males.

5.5. Movement algorithm

Analogous to the original KMA, there are three types of movement in DKA. Algorithm 1 illustrates the big male attraction movement using the edge-construction operator and the big male distraction movement using the edge-destruction operator. While

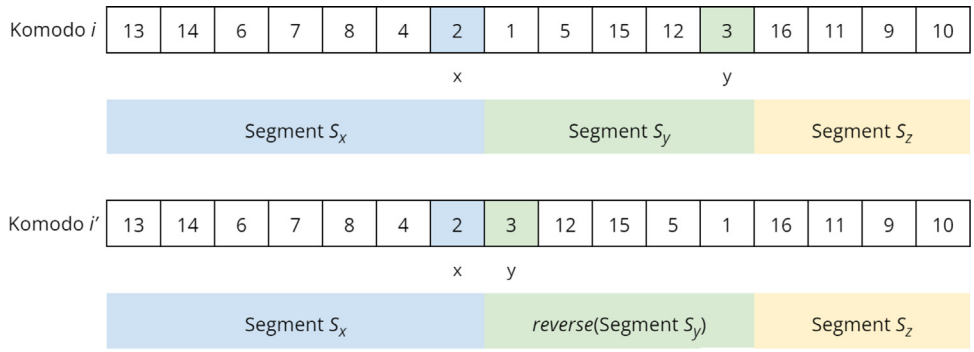


Fig. 5. Komodo i perform a 2-opt swap operator for edge $2-3$.

Algorithm 2 illustrates the female reproduction movement with an attraction movement to the highest quality big male or exploration movement, Algorithm 3 illustrates the small male movement with an exploration or attraction movement to big males. To have the same idea as moving in continuous space, DKA uses the edge-construction operator to reduce the discrete distance (Fig. 1) between tours (attraction) and uses the edge-destruction operator to increase the discrete distance between two tours.

Algorithm 1 Big Male Movement

Require: k_i, k_j a tour of big male i and big male j
Ensure: $\hat{k}_i$, new tour of big male i after movement.
  $r \leftarrow random(0, 1)$
  if $fitness(k_i) > fitness(k_j)$ && $r > 0.5$ **then**
    $\hat{k}_i \leftarrow distraction(k_i, k_j)$
  else
    $\hat{k}_i \leftarrow attraction(k_i, k_j)$
  end if

Algorithm 2 Female Movement

Require: k_i, k_j a tour of female i and big male j
Ensure: $\hat{k}_i$, new tour of female i after movement.
  $r \leftarrow random(0, 1)$
  if $r > 0.5$ **then**
    $\hat{k}_i \leftarrow attraction(k_i, k_j)$
  else
    $\hat{k}_i \leftarrow exploration(k_i)$
  end if

Algorithm 3 Small Male Movement

Require: *smep*, a small male exploration probability (0, 10)
Require: k_i, k_j a tour of small male i and big male j
Require: *PQ*, priority queue storing edges with its normalized weight and counter
Require: *extraEdges* number of extra edges to explore in the movement
Ensure: $\hat{k}_i$, new tour of small male i after movement.
  $r \leftarrow randomInt(0, 10)$
  if $r < smep$ **then**
    $\hat{k}_i \leftarrow exploration(k_i)$
  else
    $\hat{k}_i \leftarrow attraction(k_i, k_j)$
  end if

There will be a case when two komodos within a population have a similar tour, making the discrete distance between two komodos zero (Fig. 1). Zero distance causes the DKA to be unable to perform the edge-construction operation due to the unavailability of the edge to construct. To handle this case, the DKA is designed to have a minimum number of edges to be used as candidates and selected randomly in all discrete operators: edge-construction, edge-destruction, insertion, and 2-opt swap. Suppose the discrete distance between two komodo on the attraction movement is less than the minimum value. In that case, DKA will take the remaining edge(s) *extraEdges* needed from PQ with the highest priority. Using this formula, we define the discrete operator's minimum number of edges m.

$$m = \begin{cases} \max(1, \frac{t}{640}), & \text{if } bigmale \\ \max(2, \frac{t}{320}), & \text{if } female \\ \max(3, \frac{t}{160}), & \text{if } smallmale. \end{cases} \tag{16}$$

where max is a mathematical function to get the maximum value between 2 arguments, and t is the total possible edges for the TSP problem calculated using Eq. (2).

The attraction movement of a small male attracted to a big male or a big male attracted to another big male has a chance to perform an exploration movement when the discrete distance between the two komodos is less than the minimum value m. This minimum value of m tries to mimic the concept from the original KMA, where the big males can still perform low exploration. The attraction of small males will either get an edge over the big male or take the top edge from PQ randomly. So the distribution of exploitation (getting an edge from the big male) and exploration (getting an edge from PQ) will be proportionate on how different or how far in terms of discrete distance between the small male and the big male, the farther the discrete distance is, the distribution will lean more to exploitation. For exploration movement by small males, DKA will take all the edge candidates from PQ with the highest priority.

5.6. Tour finalization with 2-opt

To enhance the solution quality produced by the DKA, the DKA uses 2-opt [54] as a tour finalization. Before MaxFEs are reached, the DKA will only apply 2-opt on the best individual. So the DKA will be terminated at MaxFEs-1, and the last function evaluation will be used to evaluate the final solution after applying the 2-opt algorithm.

5.7. Comparison of DKA with other discrete algorithms

Fig. 6 summarizes the proposed Discrete Komodo Algorithm in a flowchart, and Table 1 gives the comparison of DKA with other algorithms.

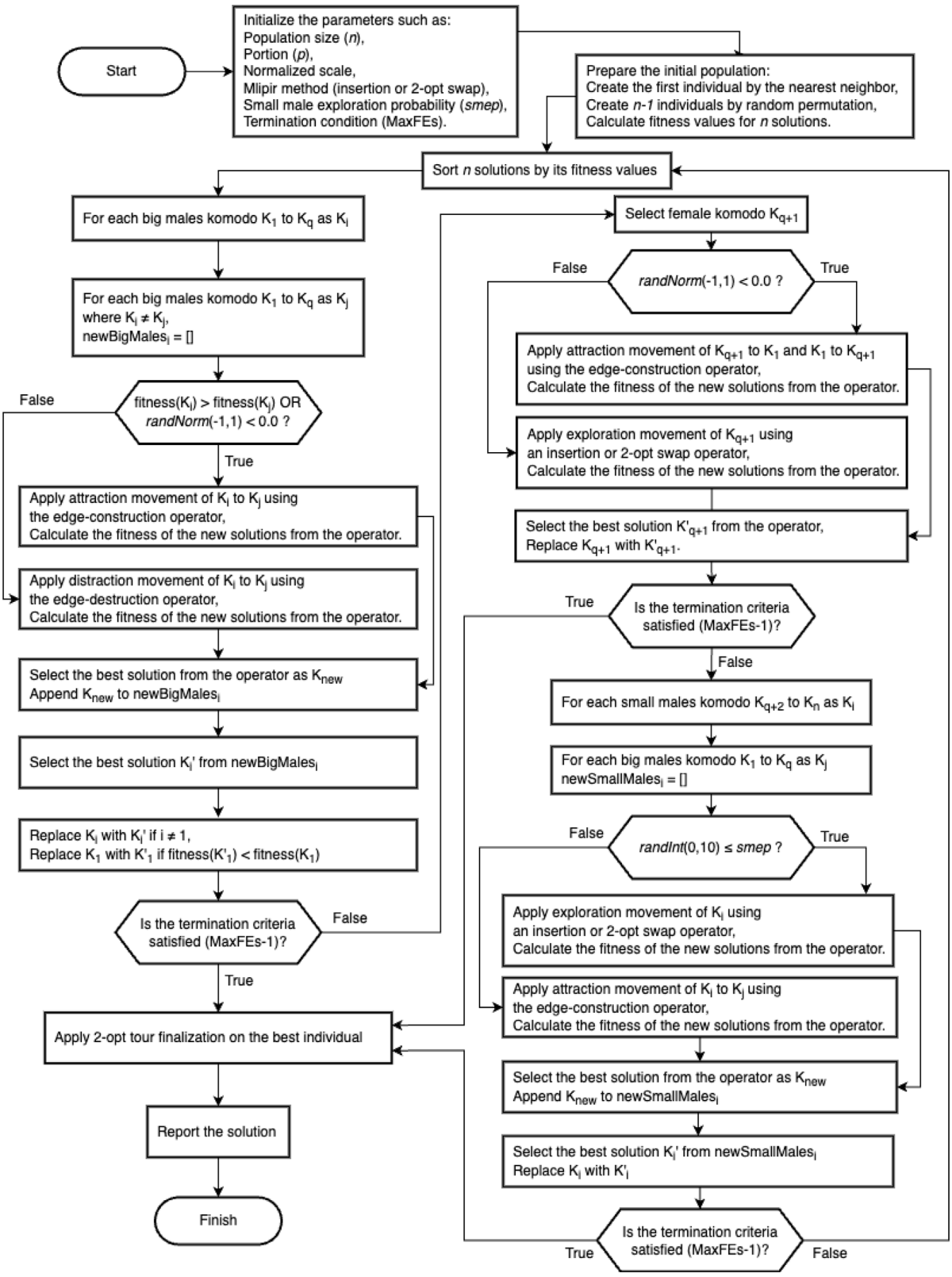


Fig. 6. The flowchart of the discrete komodo algorithm (DKA).

6. Experimental results

We tested the proposed discrete komodo algorithm (DKA) on 45 instances of symmetric TSP taken from TSPLIB [55]. The optimum route of the instances from TSPLIB is known.[1] Some literature calls it the best-known solution (BKS). We use multiple-dimensional symmetric TSP from 14 to 575 nodes to indicate the

[1] http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/STSP.html

Table 1
Comparison of DKA with DTSA and DJAYA.

Method	DTSA [48]	DJAYA [49]	DKA
Discrete distance	n/a	n/a	total different edges between two solutions
Searching strategies	The new solutions are generated from: - current tree - random tree - best tree	The new solutions are generated from: - current solution - best solution - worst solution	Split the population into three groups: - big males - female - small males The new solutions are generated from the interaction between: - two big males - female & big male - small male & big male - exploration Use priority queue to explore potential edges with dynamic probability
Discrete Operator	- swap - shift - symmetry	- swap - shift - symmetry - swap+shift - swap+symmetry - shift+symmetry - combined 1 - combined 2	*exploitation*: - edge-construction - edge-destruction *exploration*: - insertion - 2-opt swap
Parameter	- population size - search tendency - MaxFEs	- population size - search tendency 1 - search tendency 2 - MaxFEs	- population size - portion - normalized scale, - *mlipir* method - small male exploration probability - MaxFEs
Population initialization	full random, except the first individual using nearest neighbor	full random, except the first individual using nearest neighbor	full random, except the first individual using nearest neighbor
Use 2-opt finalization	yes	yes	yes
Test cases	14	14	45
Source of test problem	TSPLIB	TSPLIB	TSPLIB

Table 2
Parameter used for grid search on TSP instance.

Population size (n)	Portion (p)	Normalized scale ($\mathcal{S}$)	*Mlipir* method	*smep*
5, 10, 11, 15, 20, 21, 25, 30, 31, 35, 40, 41, 45, 50, 51, 55, 60, 61, 65, 70, 71, 75, 80, 81, 85, 90, 91, 95, 100, 101	0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9	100, 1000, 10000	insertion, 2-opt swap	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

potency of the DKA. The test problems contain information about the number of nodes; for example, berlin52 means the number of nodes is 52. Each instance has information about edge weight type that specifies how we should calculate the edge weight (or distance) between two nodes. This study incorporates four edge weight types: GEO, EXPLICIT, ATT, and EUC_2D. Section 2 on TSPLIB document[2] provides the details on how to calculate the distance for each edge weight type. Some studies like [46,50] mistakenly used Euclidean distance over TSP instances which use other edge weight types, e.g., the ulysses16 has geographical (*GEO*) edge weight type, and att48 has *ATT* edge weight type, but [46] used these problems using euclidean (*EUC_2D*) to calculate the edge weight which leads to different tour length. Some literature mentions this issue as faulty coordinate system calculation. This study excludes the faulty result for comparison with DKA.

The experiment uses the maximum number of function evaluations (MaxFEs) as a termination condition to make a fair comparison instead of the maximum number of iterations since different algorithms have different new candidate solutions generated at the end of each iteration. For statistical reliability, all the methods in the comparison are run 20 times for each configuration parameter. We present the experimental results as mean, standard deviation (SD), and Relative Error (RE). RE is the mean of the difference between the optimum cost and the mean cost of 20 runs found by the DKA. We calculate RE by using Eq. (17).

$$RE = \frac{f(mean) - f(optimum)}{f(optimum)} \times 100 \tag{17}$$

where $f(optimum)$ is the optimal/best-known solution from TSPLIB, and $f(mean)$ is the average cost of 20 runs solutions found by the algorithm. The experiment was developed on Go, specifically Go 1.17. Tests were performed on a computer with an Intel(R) Xeon(R) CPU @ 2.80 GHz, 8 GB of RAM, and Ubuntu 18.04 bionic.

[2] http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp95.pdf

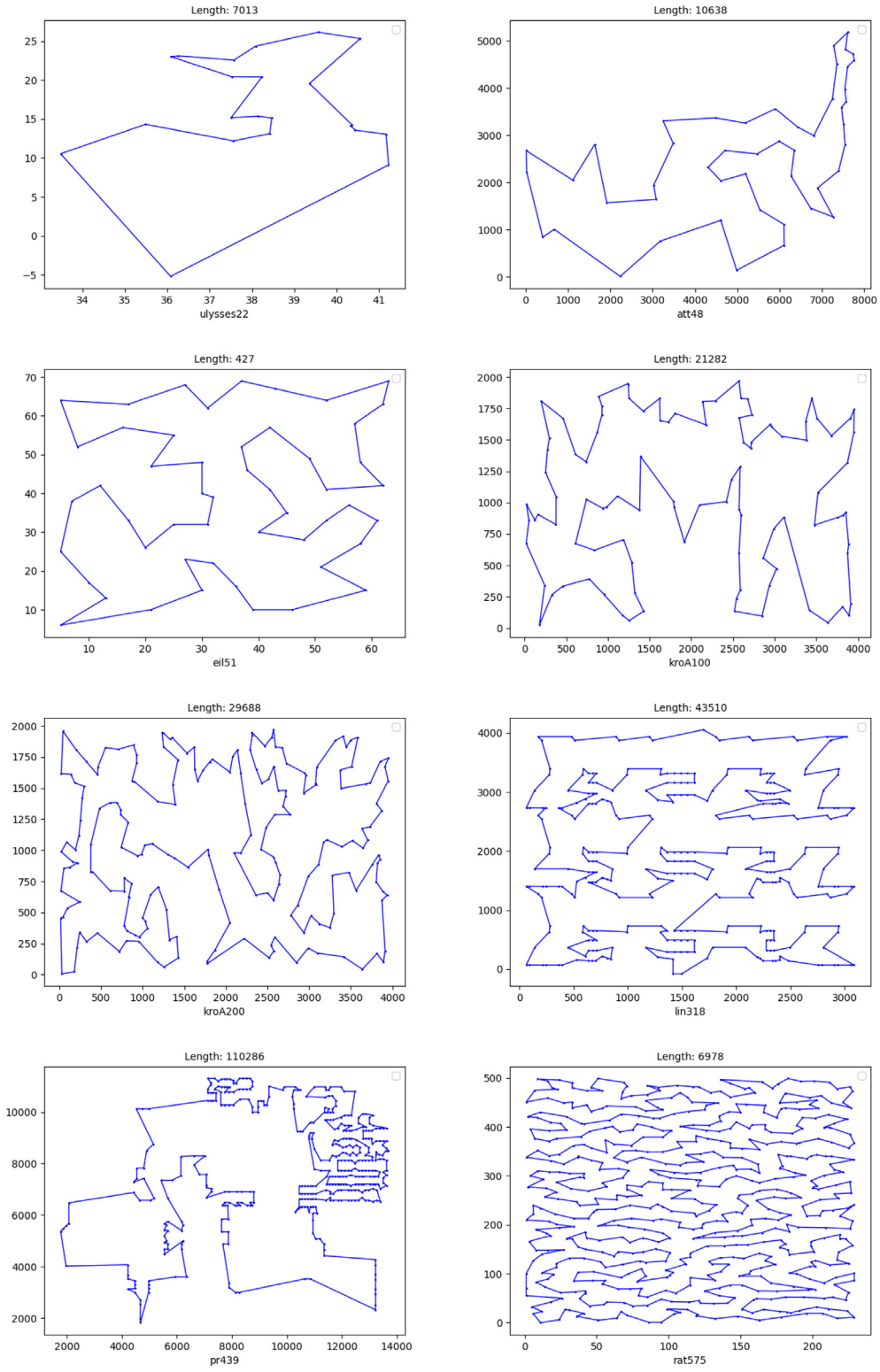


Fig. 7. The best solution computed by DKA with MaxFEs set to $D \times 500$ where D is the number of cities.

This study uses Relative Error (RE) to compare the algorithms' performance and how close the algorithm's average result is to the optimum solution. The comparison in Sections 6.4 and 6.5 utilize EUC_2D test problems with a bit different calculation for the edge weight. The algorithms on these comparisons choose not to round the calculation of the edge weight to the nearest integer. On the other hand, the guidance from TSPLIB guides to round the calculation result to the nearest integer so that the full-length tour would be an integer[3] instead of a decimal. To accommodate these comparisons, we set up the DKA to not only round the calculation result for these comparisons. Not rounding the calculation caused the total length of tour of the best-known solution to have a slightly different value between DKA and the other

[3] http://elib.zib.de/pub/mp-testdata/tsp/tsplib/stsp-sol.html

Table 3
DKA Performance over TSP problems with total runs 20 and MaxFEs *D*x500 where *D* is the number of cities.

No	Problem	MaxFEs	Optimum	Best	Worst	Mean	SD	RE (%)	Opt./Run	Mean time (ms)
1	burma14	7000	3323	3323	3323	3323	0	0	20/20	4.4
2	ulysses16	8000	6859	6859	6859	6859	0	0	20/20	10.75
3	gr17	8500	2085	2085	2085	2085	0	0	20/20	12.45
4	gr21	10500	2707	2707	2707	2707	0	0	20/20	9.4
5	ulysses22	11000	7013	7013	7013	7013	0	0	20/20	34.5
6	gr24	12000	1272	1272	1272	1272	0	0	20/20	33.75
7	fri26	13000	937	937	955	939.5	5.96	0.27	17/20	62.75
8	dantzig42	21000	699	699	713	701.35	3.94	0.34	13/20	205.6
9	att48	24000	10628	10638	10918	10755.8	85.22	1.20	0/20	244.75
10	gr48	24000	5046	5046	5187	5095.9	36.67	0.99	5/20	360.65
11	hk48	24000	11461	11461	11583	11487.1	34.14	0.23	3/20	260.7
12	rand50	25000	5553	5553	5859	5604.7	96.55	0.93	15/20	254.8
13	eil51	25500	426	427	438	429.95	3.14	0.93	0/20	262.55
14	berlin52	26000	7542	7542	7869	7646.25	97.12	1.38	8/20	191.3
15	st70	35000	675	675	691	684.1	4.81	1.35	3/20	538.35
16	eil76	38000	538	539	558	547.8	5.69	1.82	0/20	713.3
17	pr76	38000	108159	108304	114610	110134.95	1311.37	1.83	0/20	581.15
18	gr96	48000	55209	55531	57871	56279.8	567.29	1.94	0/20	962.3
19	rat99	49500	1211	1211	1243	1224.55	11.14	1.12	3/20	1030.35
20	kroA100	50000	21282	21282	21825	21436.65	143.48	0.73	2/20	1205.5
21	kroB100	50000	22141	22258	23035	22453.65	166.77	1.41	0/20	1079.55
22	kroC100	50000	20749	20769	21516	21080.95	232.09	1.59	0/20	865.35
23	kroD100	50000	21294	21398	22481	21750.2	223.87	2.14	0/20	996.15
24	kroE100	50000	22068	22146	22950	22454.7	203.42	1.75	0/20	1423.65
25	eil101	50500	629	641	664	652.6	5.31	3.75	0/20	1075.55
26	lin105	52500	14379	14426	15391	14686.65	229.62	2.14	0/20	1352.4
27	pr107	53500	44303	44303	45221	44748.25	219.43	1.01	1/20	1544.8
28	pr124	62000	59030	59164	61412	59985.5	590.62	1.62	0/20	1914.75
29	ch130	65000	6110	6201	6423	6307.9	58.11	3.24	0/20	1812.05
30	pr136	68000	96772	99692	104440	102296.6	1347.85	5.71	0/20	2100.8
31	pr144	72000	58537	58603	60961	59371.9	732.71	1.43	0/20	2688.65
32	ch150	75000	6528	6553	6898	6653.65	91.49	1.92	0/20	3210.8
33	kroA150	75000	26524	26888	27852	27358.5	314.97	3.15	0/20	2609.25
34	kroB150	75000	26130	26363	27526	26902.85	325.28	2.96	0/20	2500.45
35	pr152	76000	73682	74202	76637	75287.75	709.81	2.18	0/20	2461.8
36	kroA200	100000	29368	29688	30812	30202.5	326.46	2.84	0/20	7008.3
37	kroB200	100000	29437	30170	32037	30911.8	457.45	5.01	0/20	7669.05
38	gr202	101000	40160	41502	44057	42687.4	591.96	6.29	0/20	6677
39	tsp225	112500	3916	3989	4246	4096.05	65.08	4.59	0/20	8656.05
40	pr226	113000	80369	81168	86350	83231.6	1120.22	3.56	0/20	11217.2
41	pr264	132000	49135	49876	55577	52501.75	1390.74	6.85	0/20	13372.5
42	lin318	159000	42029	43510	45836	44379.65	566.16	5.59	0/20	33918.65
43	rd400	200000	15281	15589	16594	15996.05	225.01	4.68	0/20	31878.45
44	pr439	219500	107217	110286	118405	114640.75	2167.09	6.92	0/20	36961.95
45	rat575	287500	6773	6978	7277	7120.95	75.86	5.14	0/20	80455.55

algorithms since we implemented DKA in Go. In comparison, other researchers used MATLAB to implement their algorithms. In the end, we use the RE to compare the performance to make a fair comparison.

To find the best parameter corresponding to each problem, we conducted a Grid Search over all parameters' combination values to find the best parameter that performs best for each problem. To find the best configuration, we sorted the run results by Relative Error (RE) as defined in Eq. (17). Table 2 lists the parameters we used to perform Grid Search for each problem. With 30 different values for population size, nine for portion, three for the normalized scale, two types of *mlipir* method, and 11 for small male exploration probability, one Grid Search produces 17820 combinations of parameter settings.

We tested DKA on 45 TSP problems with MaxFEs set to $D\times500$, where D is the number of cities for the test problems and 20 total runs. DKA terminates when it finds the best-known solution or reaches MaxFEs. The best solutions of several TSP instances computed by DKA are shown in Fig. 7.

Table 3 shows the performance of DKA. *MaxFEs* is the maximum function evaluation for the termination condition of DKA. *Optimum* is the best-known solution to the problem taken from TSPLIB. *Best*, *Worst*, *Mean*, *SD* is the best, worst, average, and standard deviation of the fitness found by DKA from 20 total runs. *RE* is the relative error calculated using Eq. (17). *Opt/Run* is the number of DKA found the best-known solution from total runs, 4/20 means that from 20 runs, DKA found the best solution four times. *Mean time* (ms) is the average running time of DKA from 20 runs in milliseconds. As seen in Table 3, DKA always found the best solution for test problems burma14, ulysses16, gr17, gr21, ulysses22, and gr24.

Table 4 shows the function evaluation utilization of DKA and its best parameter settings from Grid Search. *Best FEs*, *Worst FEs*, *Mean FEs*, and *SD FEs* are the minimum, maximum, average, and standard deviation of function evaluation from 20 runs, respectively. Since the DKA terminates when it finds the best solution or reaches MaxFEs when the value of *Best FEs* or *Worst FEs* is less than *MaxFEs*, it means the DKA found the best solution and terminated early (not reached MaxFEs). As seen in Table 4, the test problem burma14, gr21, kroA200, kroB200, tsp225, rd400, and rat575 have multiple parameter settings which produced identical results. Two identical results were produced for test problem tsp225 under parameter settings: population size (n) is set to 5, the portion (p) is set to 0.3 or 0.4, normalized scale is set to 10000, *mlipir* method is set to insertion and small male exploration probability (*smep*) is set to 7.

Table 4
DKA best parameter settings for the result on Table 3.

No	Problem	MaxFEs	Best FEs	Worst FEs	Mean FEs	SD FEs	Pop. size (n)	Portion (p)	Norm. scale	*Mlipir* method	*smep*
1	burma14	7000	225	3797	1132.65	874.04	5	0.5–0.9	1000	insertion	7
2	ulysses16	8000	1806	5887	3340.35	916.95	30	0.9	1000	2-opt swap	3
3	gr17	8500	447	7077	2542.25	1617.98	10	0.7	100	insertion	9
4	gr21	10500	508	4953	2039.2	1106.64	5	0.5–0.9	1000	insertion	8
5	ulysses22	11000	6782	11000	9363.1	1070.95	35	0.9	10000	2-opt swap	2
6	gr24	12000	3252	12000	7038.5	2293.37	21	0.8	10000	2-opt swap	7
7	fri26	13000	7427	13000	11095.35	1889.27	41	0.9	10000	2-opt swap	7
8	dantzig42	21000	11510	21000	18882.05	3245.89	21	0.8	10000	insertion	7
9	att48	24000	24000	24000	24000	0	11	0.6	10000	2-opt swap	1
10	gr48	24000	10154	24000	22524.35	3547.26	11	0.6	10000	2-opt swap	8
11	hk48	24000	14855	24000	23254.8	2298.81	25	0.8	10000	2-opt swap	8
12	rand50	25000	8106	25000	19241.95	5739.13	10	0.5	10000	insertion	5
13	eil51	25500	25500	25500	25500	0	15	0.7	10000	2-opt swap	5
14	berlin52	26000	24479	26000	25923.95	331.49	21	0.8	100	2-opt swap	0
15	st70	35000	24083	35000	34069.5	2837.66	10	0.7	10000	insertion	4
16	eil76	38000	38000	38000	38000	0	11	0.7	10000	insertion	2
17	pr76	38000	38000	38000	38000	0	20	0.8	10000	2-opt swap	6
18	gr96	48000	48000	48000	48000	0	15	0.7	10000	2-opt swap	3
19	rat99	49500	23270	49500	46682.45	7035.39	10	0.6	10000	insertion	0
20	kroA100	50000	38435	50000	48960.6	3140.11	10	0.7	10000	insertion	1
21	kroB100	50000	50000	50000	50000	0	11	0.6	10000	insertion	3
22	kroC100	50000	50000	50000	50000	0	15	0.7	10000	insertion	2
23	kroD100	50000	50000	50000	50000	0	15	0.7	10000	insertion	3
24	kroE100	50000	50000	50000	50000	0	10	0.7	10000	insertion	6
25	eil101	50500	50500	50500	50500	0	15	0.7	10000	insertion	4
26	lin105	52500	52500	52500	52500	0	11	0.6	10000	insertion	3
27	pr107	53500	36643	53500	52657.15	3673.89	10	0.6	10000	insertion	4
28	pr124	62000	62000	62000	62000	0	15	0.6	10000	insertion	3
29	ch130	65000	65000	65000	65000	0	11	0.6	10000	2-opt swap	0
30	pr136	68000	68000	68000	68000	0	11	0.5	10000	insertion	3
31	pr144	72000	72000	72000	72000	0	15	0.6	10000	2-opt swap	4
32	ch150	75000	75000	75000	75000	0	10	0.6	10000	insertion	1
33	kroA150	75000	75000	75000	75000	0	10	0.7	10000	insertion	1
34	kroB150	75000	75000	75000	75000	0	11	0.6	10000	insertion	0
35	pr152	76000	76000	76000	76000	0	15	0.7	10000	insertion	3
36	kroA200	100000	100000	100000	100000	0	5	0.3–0.4	10000	insertion	2
37	kroB200	100000	100000	100000	100000	0	5	0.3–0.4	10000	insertion	7
38	gr202	101000	101000	101000	101000	0	10	0.2	10000	insertion	3
39	tsp225	112500	112500	112500	112500	0	5	0.3–0.4	10000	insertion	4
40	pr226	113000	113000	113000	113000	0	31	0.6	1000	insertion	10
41	pr264	132000	132000	132000	132000	0	11	0.2	10000	insertion	4
42	lin318	159000	159000	159000	159000	0	11	0.7	10000	insertion	10
43	rd400	200000	200000	200000	200000	0	5	0.3–0.4	10000	insertion	0
44	pr439	219500	219500	219500	219500	0	10	0.3	10000	insertion	5
45	rat575	287500	287500	287500	287500	0	5	0.3–0.4	10000	insertion	5

Table 5
A performance analysis of normalized scale on lin318 problem.

Norm. scale	Mean	SD	RE (%)
100	45288	766.02	7.75
1000	44844.8	493.69	6.69
10000	**44379.65**	**566.16**	**5.59**

Table 6
A performance analysis of *mlipir* method on lin318 problem.

Mlipir method	Mean	SD	RE (%)
insertion	**44379.65**	**566.16**	**5.59**
2-opt swap	45433.75	506.53	8.10

6.1. Convergence behavior and parameter analysis

For this analysis, we conducted a Grid Search over all combination values of parameters listed in Table 2 for test problem lin318. We use the maximum number of function evaluations (MaxFEs) set to 159000 for the termination condition. The best parameter settings from Grid Search for the lin318 problem, which are given in Table 4 is population size (n) is set to 11, the portion (p) is set to 0.7, normalized scale is set to 10000, *mlipir* method is set to insertion, and small male exploration probability (*smep*) is set to 10.

We first analyze population size (n), determining the number of komodo or solution candidates on each iteration. For this purpose, we test the varying number of n from 5 to 101 with the other parameters following the best parameter settings for lin318. From the result given in Fig. 9, population size 11 produced the best result with the lowest relative error for lin318. Table 4 also shows that the population size 10 & 11 becoming the sweet spot for most test problems and population size higher than 35 does not contribute to a better result. With the same MaxFEs, large population size will have fewer iterations and does not guarantee that the DKA could produce better solutions.

We perform the second analysis on the portion (p), which determines the proportion of big males and small males within the population. DKA will have at least two big males to make sure they have an interaction to perform an attraction movement by the edge-construction operator or distraction movement by the edge-destruction operator. Since there is no interaction between 2 small males in DKA, it is possible to have no small males within the population. So, there is no limitation to having at least two small males since the movement of small males in DKA is only attraction movement to the big males by the edge-construction operator or exploration movement by the insertion

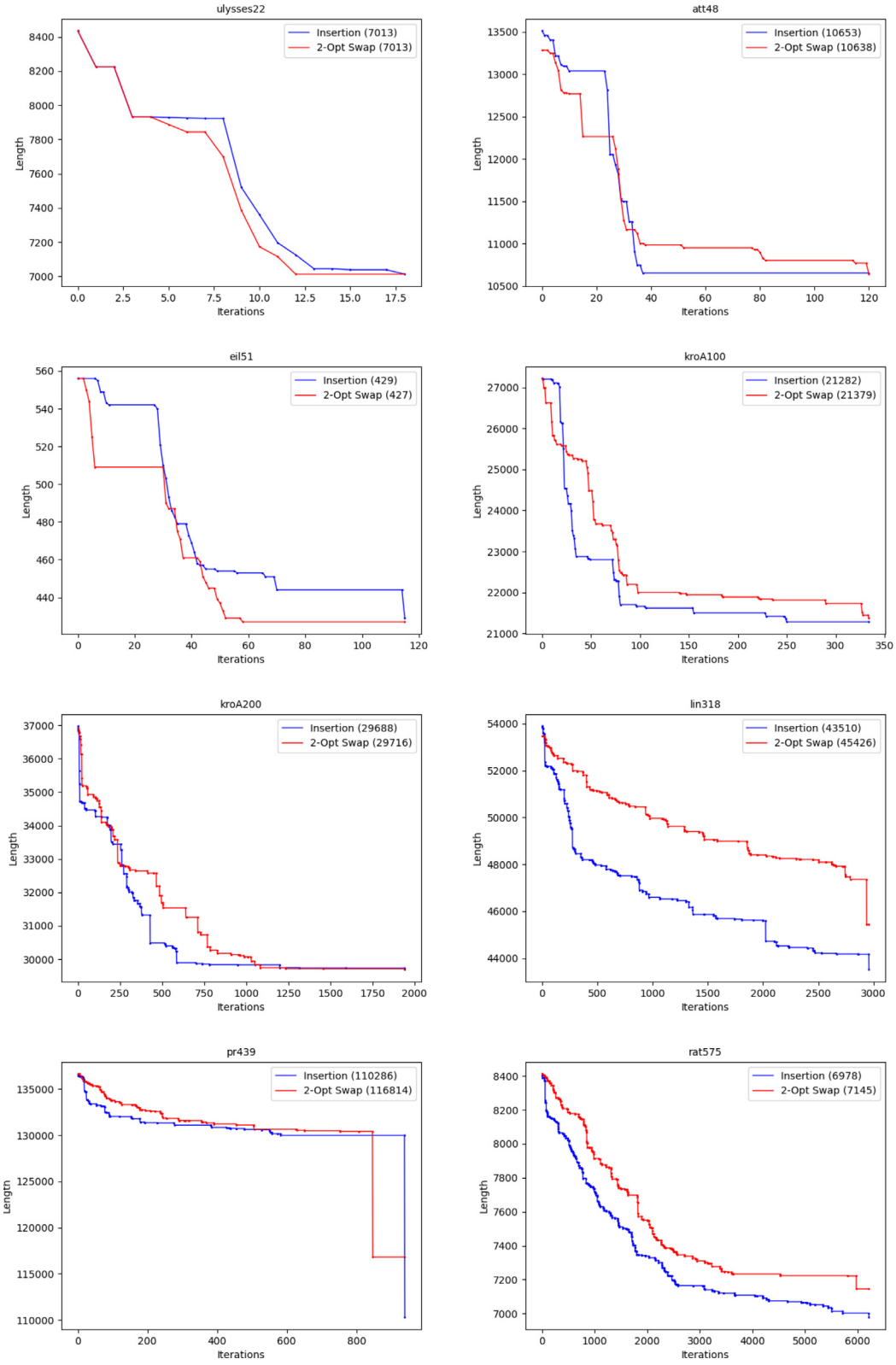


Fig. 8. The evolutionary process of the best solutions.

operator or 2-opt swap operator. No small male is attracted to another small male. Some parameter settings can produce the exact composition of komodo within a population. With five as population size, the portion (p) 0.1 and 0.2 will produce four big males, one female, and 0 small males. Portions 0.3 and 0.4 will produce three big males, one female, and one small male. Finally, the portions 0.5, 0.6, 0.7, 0.8, and 0.9 will produce two big males, one female, and two small males. This is the reason behind the best parameter settings for burma14, gr21, kroA200, kroB200, tsp225, rd400, and rat575 problems given in the Table 4 produced identical results.

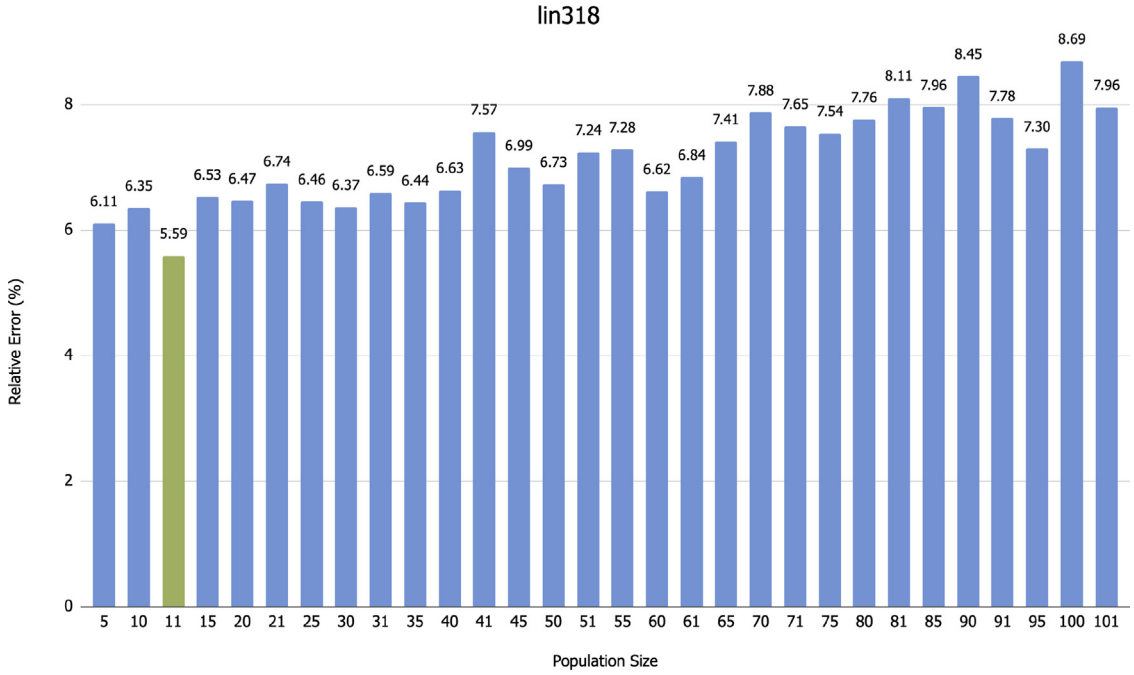


Fig. 9. A performance analysis of population size on lin318 problem.

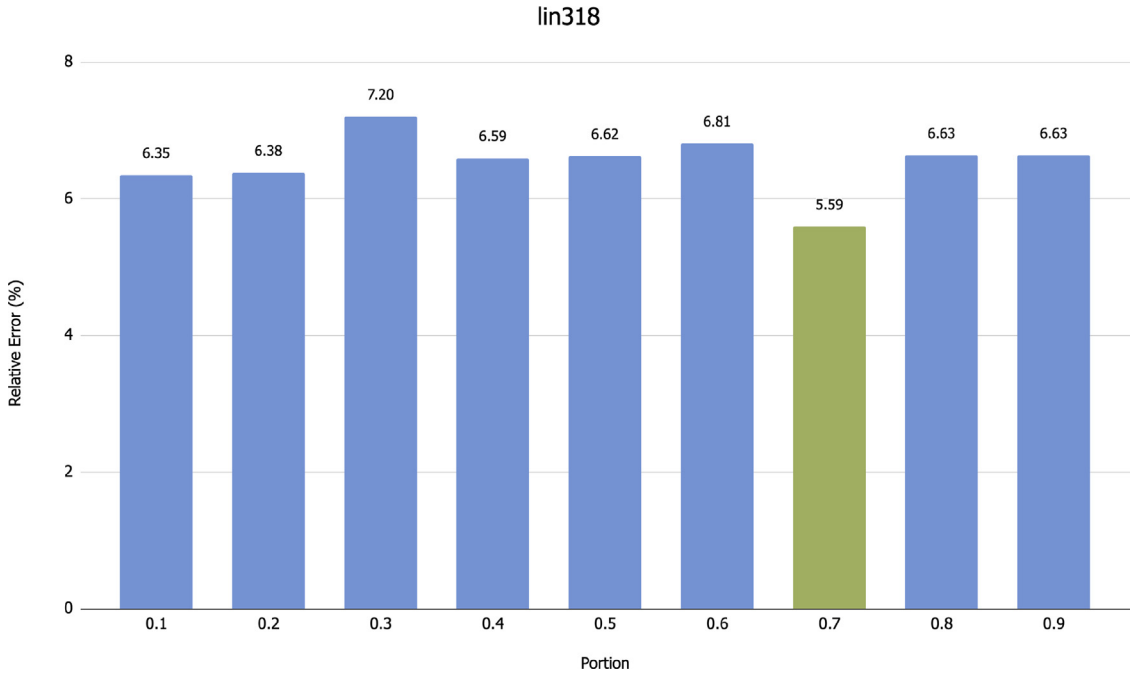


Fig. 10. A performance analysis of portion on lin318 problem.

For portion (p) parameter analysis, a varying number of p is tested from 0.1 to 0.9, with the other parameters following the best parameter settings for lin318. From the experimental results given in Fig. 10, portion 0.7 produced the best result by DKA for lin318. When examining the best parameter settings in Table 4, DKA produced the best result when the portion is higher than 0.5, which means DKA performs better when the number of small males is higher than big males within the population. Later on, the small male exploration probability (*smep*) parameter will define the behavior of the small males, whether the small males will be more exploitative by exploiting potential edges between them and big males or be more explorative by exploring high-priority edges from the priority queue.

We perform the third analysis on a normalized scale. For this purpose, we test the values for normalized scale from 100, 1000, and 10000, with the other parameters following the best parameter settings for lin318. As seen in Table 5, the higher the value of the normalized scale, the lower the relative error, which means a better result. Table 4 also confirmed that 10000 is the best-normalized scale for 39 out of 45 test problems. Normalized scale 10000 gives the DKA more chance to explore shorter edges from PQ to be constructed on tour via discrete operator before the *PV* of the shorter edges exceeds the longer edges. The lower the

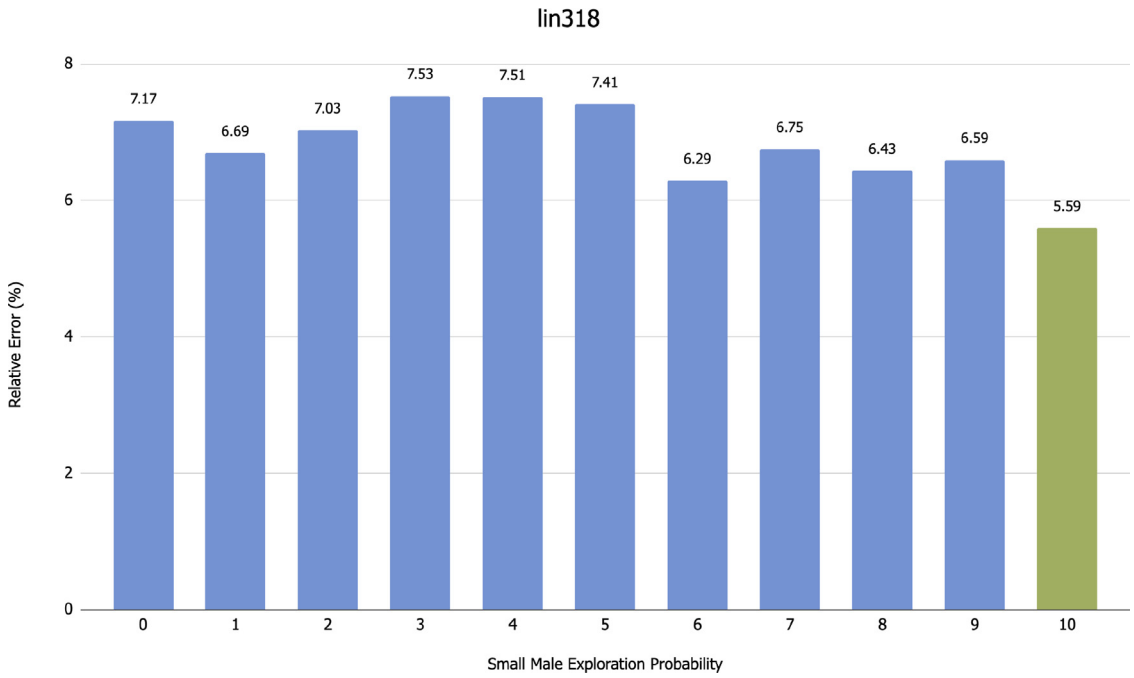


Fig. 11. A performance analysis of small male exploration probability (*smep*) on lin318 problem.

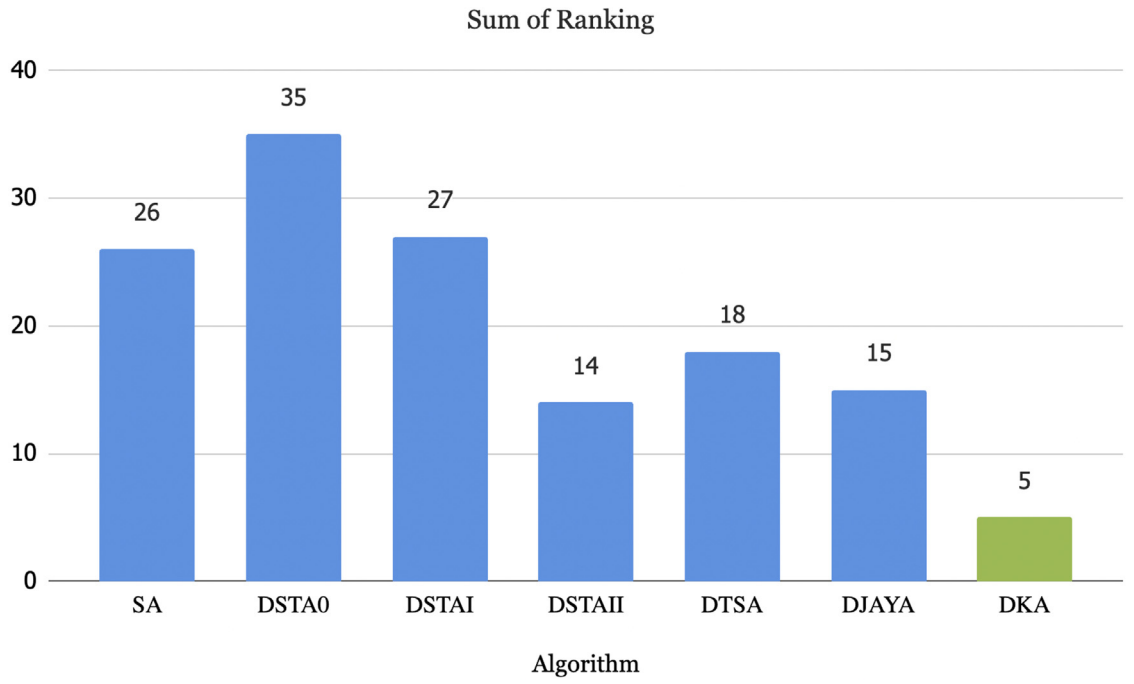


Fig. 12. Comparison of rank values for SA, DSTA variants, DTSA, DJAYA, and DKA.

value of the normalized scale, the faster the *PV* of shorter edges exceeds the longer ones.

We perform the fourth analysis on the *mlipir* method. For this purpose, 2 types of *mlipir* method are tested, insertion operator and 2-opt swap operator, with the other parameters following the best parameter settings for lin318. As seen in Table 6, the insertion operator produces better results for the lin318 problem. Table 4 also confirmed that the insertion operator is the best *mlipir* method for most of the test problems, 32 out of 45 test problems. From the convergence graph in Fig. 8, it can be seen that for the small test problems, ulysses22, att48, and eil51, the 2-opt swap operator converges quickly at the beginning iterations and get some stagnations until the insertion operator slowly catchup at the end of iterations. The insertion operator converges faster for large test problems and obtains much better solutions than the 2-opt swap operator. Hence, it can be highlighted that there are no significant differences between the insertion operator and the 2-opt swap operator for small test problems with nodes less than 100. But the insertion operator performs better for large test problems.

We perform the last analysis on the small male exploration probability (*smep*). For this purpose, a varying number of *smep* is tested from 0 to 10, with the other parameters following the best parameter settings for lin318. The result given in Fig. 11, shows

Table 7
2-opt impact on Relative Error (RE).

No	Problem	At MaxFEs-1 (before 2-opt)	At MaxFEs (after 2-opt)	RE (%) diff
1	burma14	0	0	0
2	ulysses16	0	0	0
3	gr17	0	0	0
4	gr21	0	0	0
5	ulysses22	0	0	0
6	gr24	0.18	0	0.18
7	fri26	0.43	0.27	0.16
8	dantzig42	0.84	0.34	0.50
9	att48	2.86	1.20	1.66
10	gr48	1.26	0.99	0.27
11	hk48	0.48	0.23	0.25
12	rand50	1.11	0.93	0.18
13	eil51	1.15	0.93	0.22
14	berlin52	2.98	1.38	1.60
15	st70	1.63	1.35	0.28
16	eil76	2.05	1.82	0.23
17	pr76	3.00	1.83	1.17
18	gr96	3.23	1.94	1.29
19	rat99	1.15	1.12	0.03
20	kroA100	1.46	0.73	0.73
21	kroB100	1.73	1.41	0.32
22	kroC100	2.59	1.59	1.00
23	kroD100	2.90	2.14	0.76
24	kroE100	2.41	1.75	0.66
25	eil101	5.69	3.75	1.94
26	lin105	5.29	2.14	3.15
27	pr107	1.07	1.01	0.06
28	pr124	4.29	1.62	2.67
29	ch130	4.58	3.24	1.34
30	pr136	6.78	5.71	1.07
31	pr144	2.63	1.43	1.20
32	ch150	3.04	1.92	1.12
33	kroA150	5.14	3.15	1.99
34	kroB150	5.89	2.96	2.93
35	pr152	4.93	2.18	2.75
36	kroA200	4.95	2.84	2.11
37	kroB200	6.95	5.01	1.94
38	gr202	13.21	6.29	6.92
39	tsp225	6.11	4.59	1.52
40	pr226	13.32	3.56	9.76
41	pr264	12.24	6.85	5.39
42	lin318	13.13	5.59	7.54
43	rd400	7.54	4.68	2.86
44	pr439	16.26	6.92	9.34
45	rat575	6.27	5.14	1.13

that *smep* 10 produced the best result, the lowest relative error for the lin318 problem. As seen in Table 4, no single *smep* value performs better for most test problems. Some test problems required small males to be more explorative, while others required them to be exploitative.

6.2. Impact of tour finalization by 2-opt

Table 7 shows the impact of the 2-opt algorithm applied on the best individual, which is defined in Section 5.6. It has no impact on test problems burma14, ulysses16, gr17, gr21, and ulysses22 since the discrete operator of DKA successfully found the best-known solution without applying 2-opt tour finalization. On other test problems, the intersection removal by this 2-opt could improve DKA's relative error up to 9.76%.

6.3. Comparisons with SA, DSTA variants, DTSA, and DJAYA

The first comparison is performed with Zhou et al.'s work of DSTA [47], Cinar et al.'s work of DTSA [48], and Gunduz et al.'s work of DJAYA [49]. In [47] work, kroA100, kroB100, kroC100, kroD100, and kroE100 are used to test the performance of DSTA and its variants. DSTA compares its result with SA. In [48] work, DTSA was implemented with the same parameters in [47] for a fair comparison. In [49] work, they compared DJAYA with SA, DSTA variants, and DTSA on the same problems. This study compares the proposed algorithm with SA, DSTA variants, DTSA, and DJAYA on the same problems. We directly take the comparison result of SA, and DSTA variants from [47], DTSA from [48], and DJAYA from [49]. For a fair comparison, all algorithms set maximum function evaluation (MaxFEs) at 9×10^4. Table 8 shows the comparison result for all algorithms.

Table 8
Comparison of DKA with SA, DSTA variants, DTSA, and DJAYA.

Problem	Optimum	Method	Mean	SD	RE	Rank
kroA100	21282	SA	22635.00	778.72	6.36	5
		DSTA0	23213.00	906.11	9.07	7
		DSTAI	22835.00	715.85	7.30	6
		DSTAII	21767.00	221.64	2.28	4
		DTSA	21506.78	260.55	1.06	2
		DJAYA	21705.98	290.22	1.99	3
		DKA	**21356.95**	87.52	**0.35**	1
kroB100	22141	SA	23657.00	445.78	6.85	5
		DSTA0	23794.00	517.05	7.47	7
		DSTAI	23734.00	507.38	7.19	6
		DSTAII	22880.00	302.14	3.34	2
		DTSA	23139.26	181.74	4.51	4
		DJAYA	22973.73	234.79	3.76	3
		DKA	**22378.00**	105.18	**1.07**	1
kroC100	20749	SA	22223.00	522.20	7.10	6
		DSTA0	22877.00	709.87	10.26	7
		DSTAI	21891.00	536.88	5.50	5
		DSTAII	21378.00	246.34	3.03	2
		DTSA	21817.08	217.77	5.15	4
		DJAYA	21702.02	186.32	4.59	3
		DKA	**21022.6**	187.13	**1.32**	1
kroD100	21294	SA	22911.00	483.01	7.59	5
		DSTA0	23043.00	565.80	8.21	7
		DSTAI	22665.00	592.53	6.44	4
		DSTAII	21991.00	315.32	3.27	2
		DTSA	22972.26	390.50	7.88	6
		DJAYA	22631.25	487.62	6.28	3
		DKA	**21610.35**	120.05	**1.49**	1
kroE100	22068	SA	23125.00	389.42	4.79	5
		DSTA0	23738.00	450.82	7.57	7
		DSTAI	23371.00	678.69	5.90	6
		DSTAII	22637.00	166.82	2.58	4
		DTSA	22547.00	121.96	2.17	2
		DJAYA	22582.47	252.07	2.33	3
		DKA	**22355.95**	173.65	**1.30**	1

We simulate the sum ranking of SA, DSTA variants, DTSA, DJAYA, and DKA in Fig. 12. Table 8 and Fig. 12 show that DKA is the best solver algorithm that outperformed other algorithms in the comparison for all test problems with a total rank value of 5.

6.4. Comparisons with ACO, ABC, HA, DTSA, and DJAYA

We perform the second comparison with Gündüz et al.'s work of Hierarchic Approach (HA) based on Swarm Intelligence of ACO and ABC [42], Cinar et al.'s work of Discrete Tree Seed Algorithm (DTSA) [48], and Gunduz et al.'s work of DJAYA algorithm [49]. They used ten benchmark instances in their experiment. There are oliver30, eil51, berlin52, st70, eil76, pr76, kroA100, wil101, ch150 and tsp225. This study compares the proposed algorithm with ACO, ABC, HA, DTSA, and DJAYA for the same test problems except oliver30 since we cannot find the test problem from TSPLIB. We directly take the comparison results of ACO, ABC, and HA from [42], DTSA from [48], and DJAYA from [49]. For a fair comparison, all algorithms set MaxFEs to $D \times 500$, where D is the number of cities for the test problems. Table 9 shows the comparison result of DKA with ACO, ABC, HA, DTSA, and DJAYA.

Table 9
Comparison of DKA with ACO, ABC, HA, DTSA, and DJAYA.

Problem	Method	Optimum	Mean	SD	RE	Rank
eil51	ACO	428.87	457.86	4.07	6.76	5
	ABC	428.87	590.49	15.79	37.69	6
	HA	428.87	443.39	5.25	3.39	3
	DTSA	428.87	443.93	4.04	3.51	4
	DJAYA	428.87	440.18	4.95	2.64	2
	DKA	429.98	432.08	1.96	**0.49**	1
berlin52	ACO	7544.37	7659.31	38.70	1.52	5
	ABC	7544.37	10390.26	439.69	37.72	6
	HA	7544.37	7544.37	0.00	**0.00**	1
	DTSA	7544.37	7545.83	21.00	0.02	2
	DJAYA	7544.37	7580.30	80.60	0.48	3
	DKA	7544.37	7639.08	118.06	1.26	4
st70	ACO	677.11	709.16	8.27	4.73	5
	ABC	677.11	1230.49	41.79	81.73	6
	HA	677.11	700.58	7.51	3.47	2
	DTSA	677.11	708.65	6.77	4.66	4
	DJAYA	677.11	702.30	9.56	3.72	3
	DKA	678.59	685.52	7.94	**1.02**	1
eil76	ACO	545.38	561.98	3.50	3.04	3
	ABC	545.38	931.44	24.86	70.78	6
	HA	545.38	557.98	4.10	2.31	2
	DTSA	545.38	578.58	3.93	6.09	5
	DJAYA	545.38	573.17	6.33	5.10	4
	DKA	545.38	554.79	5.24	**1.72**	1
pr76	ACO	108159.44	116321.22	885.79	7.55	5
	ABC	108159.44	205119.61	7379.16	89.65	6
	HA	108159.44	115072.29	742.90	6.39	4
	DTSA	108159.44	114930.03	1545.64	6.26	3
	DJAYA	108159.44	113258.29	1711.93	4.71	2
	DKA	108159.44	109904.81	1143.11	**1.61**	1
kroA100	ACO	21285.44	22880.12	235.18	7.49	5
	ABC	21285.44	53840.03	2198.36	152.94	6
	HA	21285.44	22435.31	231.34	5.40	4
	DTSA	21285.44	21728.40	358.13	2.08	2
	DJAYA	21285.44	21735.31	331.33	2.13	3
	DKA	21285.44	21419.32	168.28	**0.63**	1
eil101	ACO	642.31	693.42	6.80	7.96	5
	ABC	642.31	1315.95	35.28	104.88	6
	HA	642.31	683.39	6.56	6.40	3
	DTSA	642.31	689.91	4.47	7.41	4
	DJAYA	642.31	677.37	4.87	5.46	2
	DKA	642.31	662.14	6.73	**3.09**	1
ch150	ACO	6532.10	6702.87	20.73	2.61	4
	ABC	6532.10	21617.48	453.71	230.93	6
	HA	6532.10	6677.12	19.30	2.22	3
	DTSA	6532.10	6748.99	32.63	3.32	5
	DJAYA	6532.10	6638.63	52.79	**1.63**	1
	DKA	6532.28	6655.18	68.77	1.88	2
tsp225	ACO	3859	4176.08	28.34	8.22	4
	ABC	3859	17955.12	387.35	365.28	6
	HA	3859	4157.85	26.27	7.74	3
	DTSA	3859	4230.45	58.76	9.93	5
	DJAYA	3859	4095.02	42.54	6.12	2
	DKA	3859	4026.93	40.69	**4.35**	1

As seen in Table 9, under the defined configuration, the DKA produces better quality results (with the lowest relative error) for eil51, st70, eil76, pr76, kroA100, eil101, and tsp225. At the same time, the HA is the best solver for berlin52, which could produce the best-known solution. For the ch150 problem, DJAYA is slightly better than DKA. Fig. 13 shows that the DKA is the best solver algorithm for this comparison, with a total rank value of 13. The second-best solver is the DJAYA, with a total rank of 22. The most insufficient algorithm in comparisons was the ABC algorithm, with a total rank value of 54, where ABC's relative error is the highest for all test problems.

Table 10
Comparison of DKA with ACO, PSO, GA, BH, DTSA, and DJAYA.

Problem	Method	Optimum	Mean	SD	RE	Rank
eil51	ACO	428.87	461.0175	6.2974	7.4958	6
	PSO	428.87	574.8022	107.2371	34.0271	7
	GA	428.87	453.4773	9.4157	5.7377	3
	BH	428.87	458.9252	38.6365	7.008	5
	DTSA	428.87	456.5184	8.9247	6.4468	4
	DJAYA	428.87	440.4394	3.1055	2.6976	2
	DKA	429.98	441.3968	4.8327	**2.6544**	1
berlin52	ACO	7544.37	8522.9017	1152.2000	12.9703	5
	PSO	7544.37	11089.5286	2067.9323	46.9907	7
	GA	7544.37	9288.4483	1301.2108	23.1176	6
	BH	7544.37	8455.8304	508.9871	12.0813	4
	DTSA	7544.37	7761.6000	62.8594	2.8793	2
	DJAYA	7544.37	7627.0000	120.3869	**1.0952**	1
	DKA	7544.37	7952.6621	204.3979	5.4119	3
st70	ACO	677.11	757.7540	59.6079	11.9100	4
	PSO	677.11	1321.8137	269.2793	95.2140	7
	GA	677.11	1158.8458	52.1734	71.1458	6
	BH	677.11	797.5745	125.2272	17.7909	5
	DTSA	677.11	710.4037	2.7956	4.9170	3
	DJAYA	677.11	707.2151	15.3049	4.4461	2
	DKA	678.59	693.2732	3.8497	**2.1627**	1
eil76	ACO	545.38	594.1442	40.2152	8.9413	4
	PSO	545.38	975.6397	152.4061	78.8917	7
	GA	545.38	652.0593	122.0972	19.5605	5
	BH	545.38	659.1021	152.1754	20.8519	6
	DTSA	545.38	558.0623	5.7296	**2.3254**	1
	DJAYA	545.38	574.4803	5.6710	5.3357	3
	DKA	545.38	568.9725	7.8469	4.3244	2
eil101	ACO	642.31	763.9207	59.9684	18.9333	4
	PSO	642.31	1499.9911	319.7468	133.5307	7
	GA	642.31	838.8307	9.9642	30.5959	5
	BH	642.31	897.3813	210.1446	39.7115	6
	DTSA	642.31	689.8384	7.2994	7.3996	3
	DJAYA	642.31	686.8843	6.0664	6.9396	2
	DKA	642.31	675.0711	7.5416	**5.1006**	1

Table 11
Friedman's non-parametric test of SA, DSTA variants, DTSA, DJAYA, and DKA on Table 8.

	SA	DSTA0	DSTAI	DSTAII	DTSA	DJAYA	DKA
Sample size	5	5	5	5	5	5	5
Median	5	7	6	2	4	3	1
Sum of ranks	26	35	27	14	18	15	5
Mean of the ranks	5.2	7	5.4	2.8	3.6	3	1
X^2-score	25.714286						
Degree of Freedom (df)	6						
p-value	**0.0002516442654984**						

The result is very significant at $p < 0.05$.

Table 12
Holm post hoc for $1 \times N$ comparison with the DKA as the control method after Friedman test on Table 11.

	z	p	Holm	Observation
DSTA0	4.3915503283	0.000011	0.008333	$p_1 <$ Holm
DSTAI	3.2204702407	0.00128	0.01	$p_2 <$ Holm
SA	3.0740852298	0.002111	0.0125	$p_3 <$ Holm
DTSA	1.9030051422	0.05704	0.016667	$p_4 >$ **Holm**
DJAYA	1.4638501094	0.143235	0.025	
DSTAII	1.3174650985	0.187683	0.05	

6.5. Comparisons with ACO, PSO, GA, BH, DTSA, and DJAYA

We perform the last comparison with Hatamlou's Black Hole (BH) algorithm [45], Cinar et al.'s work of the Discrete Tree Seed Algorithm (DTSA) [48], and Gunduz et al.'s work of DJAYA algorithm [49]. BH was developed in 2017 and compared its performance with ACO, PSO, and GA on nine benchmark instances such as ulysses22, bayg29, att48, gr96, eil51, eil76, eil101,

Table 13
Friedman's non-parametric test of ACO, ABC, HA, DTSA, DJAYA, and DKA on Table 9.

	ACO	ABC	HA	DTSA	DJAYA	DKA
Sample size	9	9	9	9	9	9
Median	5	6	3	4	2	1
Sum of ranks	41	54	25	34	22	13
Mean of the ranks	4.555556	6	2.777778	3.777778	2.444444	1.444444
X^2-score	34.206349					
Degree of Freedom (df)	5					
p-value	**0.0000021660567506299**					

The result is very significant at $p < 0.05$.

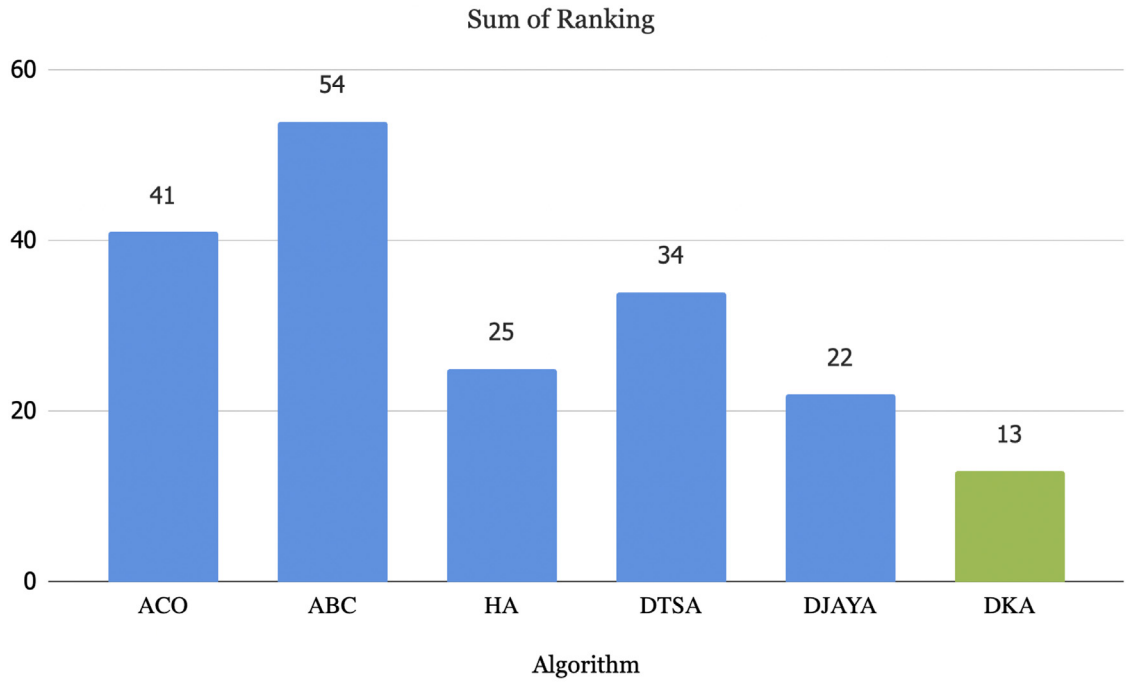


Fig. 13. Comparison of rank values for ACO, ABC, HA, DTSA, DJAYA, and DKA.

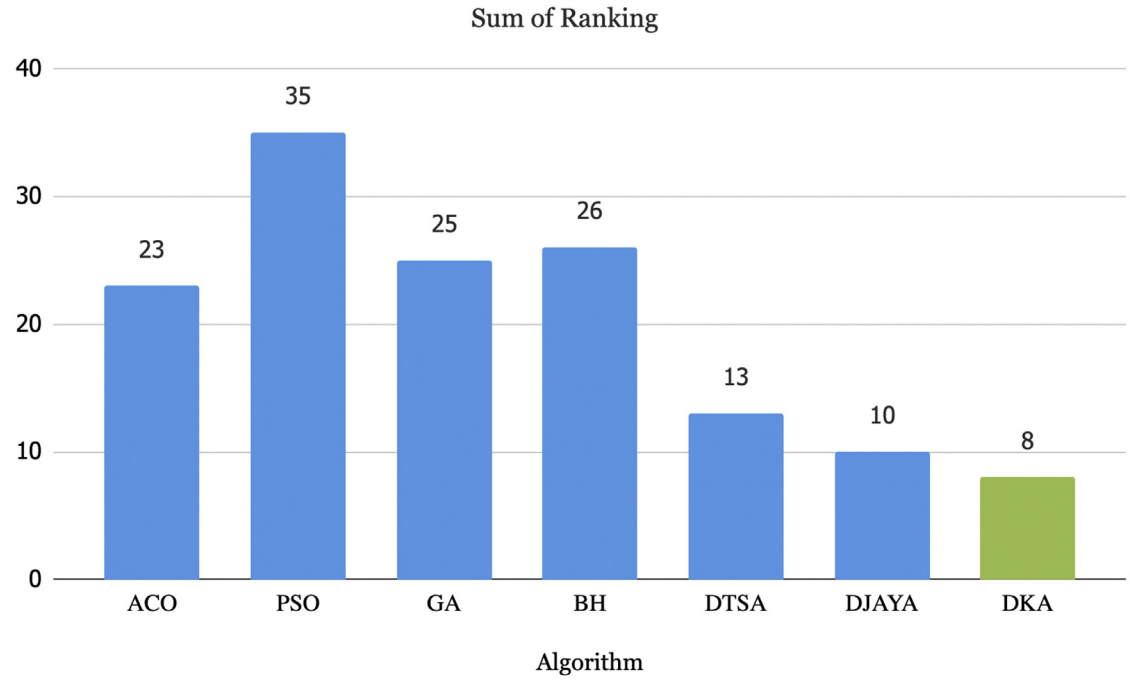


Fig. 14. Comparison of rank values for ACO, PSO, GA, BH, DTSA, DJAYA, and DKA.

Table 14
Holm post hoc for 1 × *N* comparison with the DKA as the control method after Friedman test on Table 13.

	z	p	Holm	Observation
ABC	5.1655144645	0	0.01	p_1 < Holm
ACO	3.5276684148	0.000419	0.0125	p_2 < Holm
DTSA	2.6457513111	0.008151	0.016667	p_3 < Holm
HA	1.5118578920	0.13057	0.025	p_4 > **Holm**
DJAYA	1.1338934190	0.256839	0.05	

berlin52, and st70. Three years later, Cinar et al. formulated DTSA and compared its performance against BH on eil51, berlin52, st70, wil76, and eil101. DTSA did not compare with BH on the ulysses22, bayg29, att48, and gr96 problems because of the faulty coordinate system calculation on BH. In 2021, Gunduz et al. developed DJAYA and compared its performance with the DTSA and BH on the same problems. Therefore, this study compares the proposed algorithm with ACO, PSO, GA, BH, DTSA, and DJAYA on eil51, eil76, eil101, berlin52, and st70 problems. We directly take the comparison results of ACO, PSO, GA, and BH from [45], DTSA from [48], and DJAYA from [49]. For a fair comparison, all algorithms set the maximum number of function evaluations (MaxFEs) to 2×10^4, the population size n is 100, and the total runs are 5. Table 10 gives the comparison result of DKA with ACO, PSO, GA, BH, DTSA, and DJAYA.

As seen in Table 10, the DKA produces more effective and better quality results for eil51, eil101, and st70 problems under the defined configuration parameters. While the DJAYA could produce a better result on berlin52, DTSA is the best solver algorithm for the eil76 problem. When examining the total rank values given in Fig. 14, the best solver algorithm for this comparison is DKA, with a total rank value of 8. The second best solver is the DJAYA, with a total rank value of 10. The most insufficient algorithm used in comparisons was the PSO algorithm, with a total rank value of 35.

6.6. Statistical analysis

We use a non-parametric Friedman test to verify whether there are significant differences between the algorithms in the comparisons across multiple test problems of TSP. Tables 11, 13, and 15 gives the Friedman test result for the comparison in Table 8, 9, and 10, respectively.

When we observe the result of the Friedman test in Table 11 for comparison of SA, DSTA variants, DTSA, DJAYA, and DKA, it gives an X^2-score of 25.714286, 6 as the degree of freedom, and p-value of 0.0002516442654984. The second Friedman test for comparison of ACO, ABC, HA, DTSA, DJAYA, and DKA in Table 13 gives the X^2-score of 34.206349, 5 as the degree of freedom, and p-value of 0.0000021660567506299. The last Friedman test for comparison of ACO, PSO, GA, BH, DTSA, DJAYA, and DKA in Table 15 gives the X^2-score of 25.2, 6 as the degree of freedom, and p-value of 0.00031352997647005. All the Friedman test result shows that the p-value is less than 0.05. We can conclude that there are significant differences among the algorithms. Thus DKA is the best technique as having the lowest rank.

The Friedman test shows significant differences between the algorithms in the comparisons. Thus, we conduct a post hoc

Table 15
Friedman's non-parametric test of ACO, PSO, GA, BH, DTSA, DJAYA, and DKA on Table 10.

	ACO	PSO	GA	BH	DTSA	DJAYA	DKA
Sample size	5	5	5	5	5	5	5
Median	4	7	5	5	3	2	1
Sum of ranks	23	35	25	26	13	10	8
Mean of the ranks	4.6	7	5	5.2	2.6	2	1.6
X^2-score	25.2						
Degree of Freedom (df)	6						
p-value	**0.00031352997647005**						

The result is very significant at p < 0.05.

Table 16
Holm post hoc for 1 × *N* comparison with the DKA as the control method after Friedman test on Table 15.

	z	p	Holm	Observation
PSO	3.9523952954	0.000077	0.008333	p_1 < Holm
BH	2.6349301970	0.008415	0.01	p_2 < Holm
GA	2.4885451860	0.012827	0.0125	p_3 > **Holm**
ACO	2.1957751641	0.028108	0.016667	
DTSA	0.7319250547	0.464214	0.025	
DJAYA	0.2927700219	0.769698	0.05	

procedure to identify the difference. Following the literature [56], we conduct the Holm procedure for 1 × *N* comparison with the DKA as the control method. Table 12 shows that significant differences exist between the performance of DKA compared to DSTA0, DSTAI, and SA. Table 14 shows a significant difference between the performance of DKA compared to ABC, ACO, and DTSA. Table 16 also proves that the DKA is significantly better than PSO and BH. The statistical tests confirmed that the DKA is significantly better than classical metaheuristic algorithms and comparable with state-of-the-art algorithms.

7. Conclusion and future direction

This study introduced a novel Discrete Komodo Algorithm (DKA) to solve the traveling salesman problem (TSP). The performance of DKA has been investigated on 45 different symmetric traveling salesman problems. According to the analysis and comparisons, the DKA produces better solution quality for most of the test problems. Statistical analysis shows that the proposed DKA outperformed other state-of-the-art algorithms and is significantly better than classical metaheuristic algorithms. DKA has some advantages compared to other discrete algorithms:

- DKA uses the number of different edges as a discrete distance between two solutions. This distance gives DKA visibility of potential edges as an area of exploitation between two solutions. In contrast, the other discrete algorithms do not have a clear exploitation area.
- Splitting the population into three groups enables DKA to utilize different discrete operators for exploitation and exploration. At the same time, most discrete algorithms use a single discrete operator for all movements.
- The edge-construction and edge-destruction operator allows DKA to perform an exploitation movement within the exploitation area to explore potential edges between two solutions. The edge-construction operator makes the two solutions get closer after a movement, while the edge-destruction operator increases the discrete distance between the two solutions. The swap, shift, symmetry, its combinations, and other discrete operators used by other algorithms do not guarantee that the individual is moving within an exploitation area and makes two solutions are getting closer.

- DKA uses a priority queue to explore the shorter edges with higher probability than the longer ones so that DKA can find promising edges faster. In contrast, the other discrete algorithms set up the same probability of all possible edges.

With the current design, DKA also has a limitation. DKA's discrete operators use only one edge as a parameter in one single movement. The next challenge is to find the optimal number of edges that the discrete operators can use in one single movement to accelerate the convergence rate while preventing the algorithm from early stagnation. Some of the few ideas to further research the effectiveness of DKA are as follows:

- Optimize priority queue to limit its element to promising edges only instead of storing all possible edges.
- Introduce a new parameter to control the proportion of the initial population generated by nearest neighbor vs. random permutation.
- Introduce multiple middle-quality female komodos.
- Introduce a new parameter to control whether the female komodo should be attracted to big males (exploitation) or parthenogenesis (exploration) instead of using a fixed probability of 0.5.

CRediT authorship contribution statement

Gilang Kusuma Jati: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing - Original Draft, Visualization. **Garry Kuwanto:** Methodology, Software, Formal analysis, Investigation, Writing - Original Draft. **Tahir Hashmi:** Writing - Review & Editing, Supervision. **Herman Widjaja:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

We want to thank our parents for their love and support throughout our lives. Thank you to our friends, co-workers, and teammates: Muhammad Fahrul Razi and Ivan Ferianda Ghaffar Hutagalung, for helping us set up the infrastructure to run the experiments; and all the Customer Excellence and New Retail team members who always give their best effort for Tokopedia and GoTo. We would also like to thank Fajri Koto, Paul Condylis, and the anonymous reviewers for their valuable comments and suggestions that helped improve the quality of this paper.

References

[1] K. Menger, Das botenproblem, Ergeb. Eines Math. Kolloqu. 2 (4) (1932) 11–12.
[2] G. Morton, A.H. Land, A contribution to the "travelling-salesman" problem, J. R. Stat. Soc. Ser. B Stat. Methodol. 17 (2) (1955) 185–203, https://www.jstor.org/stable/2983953.
[3] J. Robinson, On the Hamiltonian Game (A Traveling Salesman Problem), Tech. Rep., Rand Project Air Force, Arlington VA, 1949, https://apps.dtic.mil/sti/citations/AD0204961.
[4] C.O. Ozgur, J.R. Brown, A two-stage traveling salesman procedure for the single machine sequence-dependent scheduling problem, Omega 23 (2) (1995) 205–219, http://dx.doi.org/10.1016/0305-0483(94)00057-H.
[5] H. Emmons, K. Mathur, Lot sizing in a no-wait flow shop, Oper. Res. Lett. 17 (4) (1995) 159–164, http://dx.doi.org/10.1016/0167-6377(95)00008-8.
[6] Y.P. Aneja, H. Kamoun, Scheduling of parts and robot activities in a two machine robotic cell, Comput. Oper. Res. 26 (4) (1999) 297–312, http://dx.doi.org/10.1016/S0305-0548(98)00063-X.
[7] S.P. Sethi, C. Sriskandarajah, G. Sorger, J. Blazewicz, W. Kubiak, Sequencing of parts and robot moves in a robotic cell, Int. J. Flexible Manuf. Syst. 4 (3) (1992) 331–358, http://dx.doi.org/10.1007/BF01324886.
[8] G. Laporte, Modeling and solving several classes of arc routing problems as traveling salesman problems, Comput. Oper. Res. 24 (11) (1997) 1057–1061, http://dx.doi.org/10.1016/S0305-0548(97)00013-0.
[9] D.H. Smith, S. Hurley, Bounds for the frequency assignment problem, Discrete Math. 167–168 (1997) 571–582, http://dx.doi.org/10.1016/S0012-365X(96)00257-9.
[10] S.M. Allen, D. Smith, S. Hurley, Lower bounding techniques for frequency assignment, Discrete Math. 197–198 (1999) 41–52, http://dx.doi.org/10.1016/S0012-365X(99)90037-7.
[11] L.J. Hubert, F.B. Baker, Applications of combinatorial programming to data analysis: The traveling salesman and related problems, Psychometrika 43 (1) (1978) 81–91, http://dx.doi.org/10.1007/BF02294091.
[12] R.G. Bland, D.F. Shallcross, Large travelling salesman problems arising from experiments in X-ray crystallography: A preliminary report on computation, Oper. Res. Lett. 8 (3) (1989) 125–128, http://dx.doi.org/10.1016/0167-6377(89)90037-0.
[13] R.D. Plante, The nozzle guide vane problem, Oper. Res. 36 (1) (1988) 18–33, https://www.jstor.org/stable/171375.
[14] H.D. Ratliff, A.S. Rosenthal, Order-picking in a rectangular warehouse: A solvable case of the traveling salesman problem, Oper. Res. 31 (3) (1983) 507–521, http://www.jstor.org/stable/170620.
[15] R.S. Garfinkel, Minimizing wallpaper waste, part 1: A class of traveling salesman problems, Oper. Res. 25 (5) (1977) 741–751, https://www.jstor.org/stable/169475.
[16] M. Held, R.M. Karp, A dynamic programming approach to sequencing problems, J. Soc. Ind. Appl. Math. 10 (1) (1962) 196–210, https://www.jstor.org/stable/2098806.
[17] R. Bellman, Dynamic programming treatment of the travelling salesman problem, J. ACM 9 (1) (1962) 61–63, http://dx.doi.org/10.1145/321105.321111.
[18] M. Held, R.M. Karp, The traveling-salesman problem and minimum spanning trees, Oper. Res. 18 (6) (1970) 1138–1162, http://www.jstor.org/stable/169411.
[19] R. Radharamanan, L.I. Choi, A branch and bound algorithm for the travelling salesman and the transportation routing problems, Comput. Ind. Eng. 11 (1–4) (1986) 236–240, http://dx.doi.org/10.1016/0360-8352(86)90085-9.
[20] M. Padberg, G. Rinaldi, Optimization of a 532-city symmetric traveling salesman problem by branch and cut, Oper. Res. Lett. 6 (1) (1987) 1–7, http://dx.doi.org/10.1016/0167-6377(87)90002-2.
[21] H. Hernández-Pérez, J.-J. Salazar-González, A branch-and-cut algorithm for a traveling salesman problem with pickup and delivery, Discrete Appl. Math. 145 (1) (2004) 126–139, http://dx.doi.org/10.1016/j.dam.2003.09.013.
[22] C. Barnhart, E.L. Johnson, G.L. Nemhauser, M.W. Savelsbergh, P.H. Vance, Branch-and-price: Column generation for solving huge integer programs, Oper. Res. 46 (3) (1998) 316–329, http://www.jstor.org/stable/222825.
[23] B. Fleischmann, A cutting plane procedure for the travelling salesman problem on road networks, European J. Oper. Res. 21 (3) (1985) 307–317, http://dx.doi.org/10.1016/0377-2217(85)90151-1.
[24] G. Laporte, Y. Nobert, A cutting planes algorithm for the m-salesmen problem, J. Oper. Res. Soc. 31 (11) (1980) 1017–1023, http://dx.doi.org/10.2307/2581282.
[25] S.A. Cook, The complexity of theorem-proving procedures, in: Proceedings of the third annual ACM Symposium on Theory of Computing, Association for Computing Machinery, New York, 1971, pp. 151–158, http://dx.doi.org/10.1145/800157.805047.
[26] S. Arora, C. Lund, R. Motwani, M. Sudan, M. Szegedy, Proof verification and the hardness of approximation problems, J. ACM 45 (3) (1998) 501–555, http://dx.doi.org/10.1145/278298.278306.
[27] K.G. Murty, Optimization models for decision making: Volume 1, University of Michigan, Ann Arbor, 2003, http://www-personal.umich.edu/~murty/books/opti_model/.
[28] X. Geng, Z. Chen, W. Yang, D. Shi, K. Zhao, Solving the traveling salesman problem based on an adaptive simulated annealing algorithm with greedy search, Appl. Soft Comput. 11 (4) (2011) 3680–3689, http://dx.doi.org/10.1016/j.asoc.2011.01.039.
[29] Z.H. Ahmed, Improved genetic algorithms for the travelling salesman problem, Int. J. Process Manag. Benchmark. 4 (1) (2014) 109–124, http://dx.doi.org/10.1504/IJPMB.2014.059449.

[30] C. Changdar, G.S. Mahapatra, R.K. Pal, An efficient genetic algorithm for multi-objective solid travelling salesman problem under fuzziness, Swarm Evol. Comput. 15 (2014) 27–37, http://dx.doi.org/10.1016/j.swevo.2013.11.001.
[31] Y. Wang, The hybrid genetic algorithm with two local optimization strategies for traveling salesman problem, Comput. Ind. Eng. 70 (2014) 124–133, http://dx.doi.org/10.1016/j.cie.2014.01.015.
[32] A.W. Mohemmed, N.C. Sahoo, T.K. Geok, Solving shortest path problem using particle swarm optimization, Appl. Soft Comput. 8 (4) (2008) 1643–1653, http://dx.doi.org/10.1016/j.asoc.2008.01.002.
[33] B. Vallade, T. Nakashima, Improving particle swarm optimization algorithm and its application to physical travelling salesman problems with a dynamic search space, in: Applied Computing and Information Technology, Springer, 2014, pp. 105–119, http://dx.doi.org/10.1007/978-3-319-05717-0_8.
[34] M. Dorigo, V. Maniezzo, A. Colorni, Ant system: Optimization by a colony of cooperating agents, IEEE Trans. Syst. Man Cybern. 26 (1) (1996) 29–41, http://dx.doi.org/10.1109/3477.484436.
[35] M. Dorigo, L.M. Gambardella, Ant colonies for the travelling salesman problem, Biosystems 43 (2) (1997) 73–81, http://dx.doi.org/10.1016/S0303-2647(97)01708-5.
[36] S.K. Sahana, A. Jain, High performance ant colony optimizer (HPACO) for travelling salesman problem (TSP), in: International Conference in Swarm Intelligence, Springer, 2014, pp. 165–172, http://dx.doi.org/10.1007/978-3-319-11857-4_19.
[37] A. Uchida, Y. Ito, K. Nakano, Accelerating ant colony optimisation for the travelling salesman problem on the GPU, Int. J. Parallel Emergent Distrib. Syst. 29 (4) (2014) 401–420, http://dx.doi.org/10.1080/17445760.2013.842568.
[38] J.B. Escario, J.F. Jimenez, J.M. Giron-Sierra, Ant colony extended: Experiments on the travelling salesman problem, Expert Syst. Appl. 42 (1) (2015) 390–410, http://dx.doi.org/10.1016/j.eswa.2014.07.054.
[39] D. Karaboga, B. Gorkemli, A combinatorial artificial bee colony algorithm for traveling salesman problem, in: 2011 International Symposium on Innovations in Intelligent Systems and Applications, IEEE, 2011, pp. 50–53, http://dx.doi.org/10.1109/INISTA.2011.5946125.
[40] M.S. Kıran, H. İşcan, M. Gündüz, The analysis of discrete artificial bee colony algorithm with neighborhood operator on traveling salesman problem, Neural Comput. Appl. 23 (1) (2013) 9–21, http://dx.doi.org/10.1007/s00521-011-0794-0.
[41] M. Malek, M. Guruswamy, M. Pandya, H. Owens, Serial and parallel simulated annealing and tabu search algorithms for the traveling salesman problem, Ann. Oper. Res. 21 (1) (1989) 59–84, http://dx.doi.org/10.1007/BF02022093.
[42] M. Gündüz, M.S. Kiran, E. Özceylan, A hierarchic approach based on swarm intelligence to solve the traveling salesman problem, Turk. J. Electr. Eng. Comput. Sci. 23 (1) (2015) 103–117, http://dx.doi.org/10.3906/elk-1210-147.
[43] M. Mahi, Ö.K. Baykan, H. Kodaz, A new hybrid method based on particle swarm optimization, ant colony optimization and 3-opt algorithms for traveling salesman problem, Appl. Soft Comput. 30 (2015) 484–490, http://dx.doi.org/10.1016/j.asoc.2015.01.068.
[44] S. Suyanto, A.A. Ariyanto, A.F. Ariyanto, Komodo mlipir algorithm, Appl. Soft Comput. 114 (2022) 108043, http://dx.doi.org/10.1016/j.asoc.2021.108043.
[45] A. Hatamlou, Solving travelling salesman problem using black hole algorithm, Soft Comput. 22 (24) (2018) 8167–8175, http://dx.doi.org/10.1007/s00500-017-2760-y.
[46] Y. Chunhua, T. Xiaolin, Z. Xiaojun, G. Weihua, State transition algorithm for traveling salesman problem, in: Proceedings of the 31st Chinese Control Conference, IEEE, 2012, pp. 2481–2485, https://ieeexplore.ieee.org/document/6390342.
[47] X. Zhou, D.Y. Gao, C. Yang, W. Gui, Discrete state transition algorithm for unconstrained integer optimization problems, Neurocomputing 173 (2016) 864–874, http://dx.doi.org/10.1016/j.neucom.2015.08.041.
[48] A.C. Cinar, S. Korkmaz, M.S. Kiran, A discrete tree-seed algorithm for solving symmetric traveling salesman problem, Eng. Sci. Technol. 23 (4) (2020) 879–890, http://dx.doi.org/10.1016/j.jestch.2019.11.005.
[49] M. Gunduz, M. Aslan, DJAYA: A discrete Jaya algorithm for solving traveling salesman problem, Appl. Soft Comput. 105 (2021) 107275, http://dx.doi.org/10.1016/j.asoc.2021.107275.
[50] G.K. Jati, R. Manurung, Suyanto, Discrete firefly algorithm for traveling salesman problem: A new movement scheme, in: Swarm Intelligence and Bio-Inspired Computation, Elsevier, 2013, pp. 295–312, http://dx.doi.org/10.1016/B978-0-12-405163-8.00013-2.
[51] R. Matai, S. Singh, M.L. Mittal, Traveling salesman problem: An overview of applications, formulations, and solution approaches, in: D. Davendra (Ed.), Traveling Salesman Problem, IntechOpen, Rijeka, 2010, http://dx.doi.org/10.5772/12909.
[52] R.M. Karp, On the computational complexity of combinatorial problems, Networks 5 (1) (1975) 45–68, http://dx.doi.org/10.1002/net.1975.5.1.45.
[53] K. Karagül, E. Aydemir, S. Tokat, Using 2-opt based evolution strategy for travelling salesman problem, Int. J. Optim. Control Theor. Appl. (IJOCTA) 6 (2) (2016) 103–113, http://dx.doi.org/10.11121/ijocta.01.2016.00268.
[54] G.A. Croes, A method for solving traveling-salesman problems, Oper. Res. 6 (6) (1958) 791–812, http://www.jstor.org/stable/167074.
[55] G. Reinelt, TSPLIB–a traveling salesman problem library, ORSA J. Comput. 3 (4) (1991) 376–384, http://dx.doi.org/10.1287/ijoc.3.4.376.
[56] N.A.A. Aziz, M. Mubin, Z. Ibrahim, S.W. Nawawi, Statistical analysis for swarm intelligence—simplified, Int. J. Future Comput. Commun. 4 (3) (2015) 193–197, http://dx.doi.org/10.7763/IJFCC.2015.V4.383.